

National Emission Standards for Hazardous Air Pollutants (NESHAP) Maximum Achievable Control Technology (MACT) Floor Analysis for Example Industry Units for Final Rule

OAR-OCAP-NRD-EDAB and/or author names here

November 20, 2025

Introduction

Section 112 of the Clean Air Act (CAA) requires the the U.S. Environmental Protection Agency (EPA) establish National Emission Standards for Hazardous Air Pollutants (NESHAP) for the control of the hazardous air pollutants (HAP) emitted from both new and existing sources in a source category. These standards must reflect the maximum degree of reduction in emissions that is achievable. The minimum level of control is referred to as the “Maximum Achievable Control Technology (MACT) floor.” The method for determining the MACT floor as the minimum control level allowed for establishing standards for a NESHAP is defined for both new and existing sources by CAA Section 112(d)(3). For new sources, the MACT floor cannot be less stringent than the emission control that is achieved in practice by the best-controlled similar source. For existing sources, the MACT floor cannot be less stringent than the average emission limitation achieved by the best-performing 12 % of existing sources for source categories with 30 or more sources or those governed by CAA Section 129(a)(2), or the best-performing 5 sources for source categories with fewer than 30 sources for those governed by CAA Section 112(d)(3).

The purpose of this memorandum is to present the methodology and the results of the MACT floor analysis for the NESHAP source category Example Industry. This MACT floor analysis uses data collected in a nationwide survey of Example Industry unit owners and operators conducted by the EPA in 2025 under the Information Collection Request for NESHAP for Example Industry.

MACT Floor Analysis Hazardous Air Pollutants

The specific chemicals, compounds, or groups of compounds designated to be HAP are listed in *CAA section 112(b)*. From this HAP list, the following HAP, among others, were identified to be emitted from Example Industry units and were included in the MACT floor analysis: Chemical X, Chemical Y.

MACT Floor Analysis subcategories

Under *CAA section 112(d)(1)*, the EPA has the discretion to “...distinguish among classes, types, and sizes of sources within a category or subcategory in establishing...” standards. When separate subcategories are established, a MACT floor is determined separately for each subcategory. To determine whether the EGU source category warrants subcategorization for the MACT floor analysis, the EPA reviewed Example Industry design, operating information, and air emissions data compiled in the ICR data set and other information collected by the Agency for development of the NESHAP for this source category. Based on this review, the EPA concluded that there are significant design and operational differences in Example Industry that affect the levels of emissions of certain HAP to justify subcategorizing Example Industry for the purpose of establishing MACT floors.

For the MACT floor analysis, the following Example Industry subcategories were defined:

- **HAP Chemical X: Subcategory, Size:** *describe it here and how/why it was determined*
Facility size was determined as greater or less than ... and indicated by 'A' for the first set and 'B' for the second set. Since subcategory 'B' is a new designation, both existing and new source MACT floor analyses were needed. For facility size 'A' only existing source MACT floor analyses were calculated.
- **HAP Chemical Y: No subcategory** *describe it here and how/why it was determined*
The facility size subcategory does not affect HAP Chemical Y, no subcategory differentiation was applied.

Calculating Emission Values used in the MACT Floor Analysis

The EPA received emissions data from industry respondents under the Example Industry MACT ICR in a number of different units of measure and converted these units of measure to a common ppm, ppm, ppm, mg/m³/s for Chemical X, Chemical X, Chemical X, Chemical Y respectively.

If any outlier analysis or subsequent QA/QC was done on the emissions data prior to MACT floor analysis, please include it here.

MACT Floor Analysis Methodology

The first step in the MACT floor analysis for each subcategory and HAP (or surrogate) was to rank all of the Example Industry units in the subcategory (for which there were emissions data in the ICR data set) for the pollutant type by emission level (lowest to highest). From this ranking, a MACT floor pool of sources was identified for determining the minimum control level allowed for the MACT floor, consistent with the criteria defined for new and existing sources by CAA Section 112(d)(3). For the new-source MACT floors, the best-controlled similar source was identified for which there were individual source test run data in the ICR data set. The selection of top performing sources for the existing source MACT floor was performed using the `MACT_existing()` function in the `UPLforOAR` R package version 0.1.0, with details in Appendix B. The selection of the best performing source for the new source MACT floor was performed using the `MACT_new()` function in the `UPLforOAR` R package version 0.1.0, with details in appendix B.

For the existing-source MACT floors, selection of the MACT floor pool size (i.e., number of units to be included in the determination of the average emission limitation value) was determined on an individual subcategory basis as described below.

- **Subcategory 1.** *do any subcategories need different handling?*
- **Subcategory 2.** *do any subcategories need different handling?*

Probability distribution Selection

The next step in the MACT floor analysis was to incorporate data variability into the calculations of the applicable MACT floor limits for the subcategories using an upper prediction limit (UPL) approach. Specifically, the MACT floor limit is an UPL calculated with the Student's t-distribution using the `qt()` function in the `R stats` software. The Student's t-distribution has also been used in other EPA rulemakings (e.g., NESHAP for Portland Cement, NSPS for Hospital/Medical/Infectious Waste Incinerators [HMIWI], NESHAP for Industrial, Commercial, and Institutional Boilers and Process Heaters) in accounting for variability and reflects the level of confidence. The level of confidence represents the level of protection afforded to facilities whose emissions are in line with the best performers, and consequently, the level of confidence is not arbitrary. For example, with significance of $\alpha = 0.99$, a 99% level of confidence means that a facility whose emissions are in line with the best performers has one chance in 100 of exceeding the floor limit. A prediction interval for a single future observation (or an average of several test observations) is an interval that will, with a specified degree of confidence, contain the next (or the average of some other pre-specified number) of randomly selected observation(s) from a population. In other words, the UPL estimates what the upper bound of future values will be, based upon present or past background samples taken. The UPL consequently represents the value at which we can expect the mean of future observations for the HAP or HAP surrogate emissions to fall within a specified level of confidence, based upon the measurements from an

independent sample from the same population. The UPL approach encompasses all the data point-to-data point variability.

The form of the UPL equation differs depending upon the nature of the data set to which it is applied. To this end, the data sets were evaluated for each HAP and HAP surrogate to ascertain whether the data were distributed normally, log-normally, or skew-normally. Statistical tests of the kurtosis and skewness are then used to evaluate the normality assumption. The skewness statistic, S , characterizes the degree of asymmetry of a given data distribution, and is calculated according to the Fisher method as:

$$S = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma} \right)^3$$

Where n is the sample size, $\bar{x}$ is the mean of emissions data, and σ is the standard deviation (Doric et al. 2007). Normally distributed data have $S = 0$. An S that is greater (less) than 0 indicates that the data are asymmetrically distributed with a right (left) tail extending towards positive (negative) values. The standard error of the skewness statistic, SES , can be approximated by $SES = \sqrt{\frac{6n(n-1)}{(n-2)(n+1)(n+3)}}$, where n is the sample size. The kurtosis statistic, K , characterizes the degree of peakedness or flatness of a given data distribution in comparison to a normal distribution, and is calculated according to the Fisher method as:

$$K = \frac{\sum_{i=1}^n (x_i - \bar{x})^4}{(n-1)\sigma^4} - 3$$

for small data sets where $n = 3$ and as:

$$K = \frac{n(n+1)}{(n-1)(n-2)(n-3)} \sum_{i=1}^n \left(\frac{x_i - \bar{x}}{\sigma} \right)^4 - \frac{3(n-1)^2}{(n-2)(n-3)}$$

for larger data sets. Normally distributed data have $K = 3$. A K that is greater (less) than 3 indicates a relatively peaked (flat) distribution. For small data sets where $n = 3$, the standard error of the kurtosis statistic, SEK , can be approximated by $SEK = \sqrt{24/n}$, where n is the sample size. Otherwise, $SEK = \sqrt{\frac{24n(n^2-1)}{(n-2)(n+3)(n-3)(n+5)}}$.

For each data set to which the UPL was applied (i.e., the separate data sets for each HAP and HAP surrogate type applicable to the subcategory), the S and K statistics were calculated using both the reported test run values and the log-transformed reported test run values. For small data sets where $n = 3$, if $|\frac{S}{SES}|$ for the non-transformed data is less than $|\frac{S}{SES}|$ for the log-transformed data then the distribution is considered normal, otherwise it is considered log-normal. The skew-normal distribution can only be implemented for data where $n > 3$. Generally, if $|\frac{S}{SES}|$ or $|\frac{K}{SEK}|$ are greater than $\text{qnorm}(0.975, 0, 1)$ (the 97.5th quantile of the standard normal, which is approximately 1.96) the data are considered non-normal. Using these ratios, a series of decisions is used to determine the best distribution choice. First, each ratio of $|\frac{S}{SES}|$ and $|\frac{K}{SEK}|$ are calculated, and if they are less than 1.96 for untransformed or log-transformed, the data are tentatively assigned as normal. If both the skewness and kurtosis ratios came back normal for the log-transformed data and the untransformed data, then we check: if $|\frac{S}{SES}|$ is smaller for the untransformed data than the log-transformed data, the distribution is finalized as normal, if it is larger than the distribution is finalized as log-normal. For all other circumstances where both the skewness and kurtosis ratios for the untransformed data were normal, the final distribution choice is normal. Similarly, for all other circumstances where both the skewness and kurtosis ratios for the log-transformed data were normal, the final distribution is log-normal. All other circumstances are designated for the skew-normal distribution. The distribution selection is implemented via the function `distribution_type()` in UPLforOAR version 0.1.0, which is documented in Appendix B.

UPL Calculation for Normal Data

In the case of normally distributed data, the MACT floor limit is an UPL calculated as:

$$UPL = \bar{x} + t_{df, \alpha} \sqrt{\sigma^2 \left(\frac{1}{n} + \frac{1}{m} \right)}$$

Where $\bar{x}$ is the mean of the top performing sources calculated as $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$, n is the number of test runs, m is the number of future test runs, σ^2 is the variance calculate as $\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$, df is the degrees of freedom calculated as $df = n - 1$, and $t_{df, \alpha}$ is the quantile of the t -distribution with df degrees of freedom at α significance. This UPL calculation was performed in **R** using the `Normal_UPL()` function from the **UPLforOAR** package version 0.1.0. See Appendix B for the function details.

UPL Calculation for Log-normal Data

In the case of log-normally distributed data, the MACT floor limit is an UPL calculated from a log-normal approximation described in Bhaumik and Gibbons, 2004 using the Gram-Charlier Type-A Series expansion.

$$UPL = e^{\hat{\mu} + \frac{\hat{\sigma}^2}{2}} + \frac{z_{\alpha}}{m} \sqrt{m e^{2\hat{\mu} + \hat{\sigma}^2} (e^{\hat{\sigma}^2} - 1) + m^2 e^{2\hat{\mu} + \hat{\sigma}^2} \left(\frac{\hat{\sigma}^2}{n} + \frac{\hat{\sigma}^4}{2(n-1)} \right)}$$

Where n is the number of test runs, m is the number of future test runs, $\hat{\mu} = \frac{1}{n} \sum_{i=1}^n \log(x_i)$ is the mean of the log-transformed data from the top performing sources, $\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (\log(x_i) - \hat{\mu})^2$ is the variance of the log-transformed data, and z_{α} is the percentile of the log-normal distribution at α significance estimated using the trapezoidal rule from equation 26 in Bhaumik and Gibbons 2004:

$$\int_0^{z_{\alpha}} \left| \left(1 - \frac{\sqrt{\beta_{1z}}}{6} (3z - z^3) + \frac{(\beta_{2z-3})(3 - 6z^2 + z^4)}{24} \right) \phi(z) \right|$$

Where $\phi(z)$ is the standard normal and z is a sequence from -5 to 5 by intervals of 0.101 . The coefficients are calculated following equation 25 as:

$$\beta_{2z} = \frac{e^{4\hat{\sigma}^2} + 2e^{3\hat{\sigma}^2} + 3e^{2\hat{\sigma}^2} - 3}{m(e^{\hat{\sigma}^2} - 1)^2} + 3 \left(1 - \frac{1}{m} \right)$$

and:

$$\sqrt{\beta_{1z}} = \frac{\sqrt{e^{\hat{\sigma}^2} - 1}(e^{\hat{\sigma}^2} + 2)}{\sqrt{m}}$$

This UPL calculation was performed in **R** using the `Lognormal_UPL()` function from the **UPLforOAR** package version 0.1.0. See Appendix B for the function details.

UPL Calculation for Skew-normal Data

The same UPL equation as for normal data is used, but the t -statistic is substituted for a new t -statistic that is iteratively adjusted for the skewed distribution until it achieves α level of significance. To achieve this, an approximation of the incomplete beta function, is used:

$$I_{u_0}(a, b) \approx \frac{\Gamma(a, w)}{\Gamma(a)} + \frac{e^{-w} w^a}{\Gamma(a)} \left(\frac{a-1-w}{2b} + \frac{1}{(2b)^2} \left(\frac{a^3}{2} - \frac{5a^2}{3} + \frac{3a}{2} - \frac{1}{3} - w \left(\frac{3a^2}{2} - \frac{11a}{6} + \frac{1}{3} \right) + w^2 \left(\frac{3a}{2} - \frac{1}{6} \right) - w^3 \frac{1}{2} \right) \right)$$

Where u_0 and w are defined as:

$$u_0 = \frac{1}{1 + \frac{t_{new}^2}{n-1}}, \quad w = b \left(\frac{u_0}{1 - u_0} \right)$$

Where n is the number of test runs and t_{new} is the iteratively adjusted t -statistic. Six expanded terms of the incomplete beta function are calculated with (a_i, b_i) pairs:

$$b_1, \dots, b_6 = \frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}, 1, 1$$

and:

$$a_1, \dots, a_6 = \frac{n-1}{2}, \frac{n+1}{2}, \frac{n+3}{2}, \frac{n+5}{2}, \frac{n-1}{2}, \frac{n+1}{2}$$

These six incomplete beta function terms, $I_{u_0}(a_i, b_i)$, for $i \in (1, 6)$, are then used in the subsequent calculation of the probability $P(t_{new})$ given the trial t -statistic t_{new} and actual skewness S , and kurtosis K :

$$\begin{aligned} P(t_{new}) = & 1 - \frac{I_{u_0}(a_1, b_1)}{2} - S \times \left(I_{u_0}(a_5, b_5) \times \left(\frac{2n-1}{6\sqrt{2n\pi}} \right) - I_{u_0}(a_6, b_6) \times \left(\frac{n-1}{3\sqrt{2n\pi}} \right) \right) + \\ K \times & \left(I_{u_0}(a_1, b_1) \times \left(\frac{n-1}{24} \right) - I_{u_0}(a_2, b_2) \times \left(\frac{(n-1)(n+2)}{12n} \right) + I_{u_0}(a_3, b_3) \times \left(\frac{(n+4)(n-1)}{24n} \right) \right) - \\ S^2 \times & \left(I_{u_0}(a_1, b_1) \times \left(\frac{(n-1)(2n+5)}{72} \right) - I_{u_0}(a_2, b_2) \times \left(\frac{(n-1)(2n^2+5n+8)}{24n} \right) - \right. \\ I_{u_0}(a_3, b_3) \times & \left. \left(\frac{(n-1)(2n^2+5n+12)}{24n} \right) - I_{u_0}(a_4, b_4) \times \left(\frac{(n-1)(2n^2+5n+12)}{72n} \right) \right) \end{aligned}$$

Where S and K are the skewness and kurtosis estimates defined earlier.

First, the updated probability is calculated using the original t -statistic after accounting for skewness and kurtosis. If this $P(t)$ is within 0.0001 of α , then the UPL is calculated with the original t -statistic as:

$$UPL = \bar{x} + t_{df, \alpha} \sqrt{\sigma^2 \left(\frac{1}{n} + \frac{1}{m} \right)}$$

Which is the same as the UPL for normally distributed data. If, however, the updated probability $P(t)$ is more than 0.0001 different than α , then a list of 20,000 potential t_{new} are proposed in increasing or decreasing increments of 0.0001 (depending on whether $P(t)$ is less than or greater than α). The first t_{new} such that $P(t_{new})$ is within 0.0001 of α is selected as the new t -statistic to use in the UPL calculation for skew-normal data:

$$UPL = \bar{x} + t_{new} \sqrt{\sigma^2 \left(\frac{1}{n} + \frac{1}{m} \right)}$$

This UPL calculation was performed in R using the `Skewed_UPL()` function from the `UPLforOAR` package version 0.1.0. See Appendix B for the function details.

Adjustment for Below Detection Level Emissions Data

To account for the effect of measurement imprecision associated with the data in the ICR databases (that include method detection level data), a protocol specified by the EPA was used. The procedure for determining a representative detection level (RDL) begins with identifying all of the available reported pollutant-specific method detection levels for the best performing units regardless of any subcategory (e.g., existing or new, fuel type, etc.). From that combined pool of data, we calculate the arithmetic mean value. By limiting the data set to those tests used to establish the floor or emissions limit (i.e., best performers), the EPA believes that the result is representative also of the best performing testing companies and laboratories. The EPA also believes that the outcome should minimize the effect of a test(s) with an inordinately high method detection level (e.g., the sample volume was too small, the laboratory technique was insufficiently sensitive, or the procedure for determining the detection level was other than that specified). The EPA then calls the resulting mean of the method detection levels the RDL, and the Agency considers it characteristic of accepted source emissions measurement performance.

The second step in the process is to calculate three times the RDL to compare with the calculated floor or emissions limit. We use the multiplication factor of 3. For comparing to the floor, if three times the RDL were less than the calculated floor or emissions limit (e.g., calculated from the UPL), we would conclude that measurement variability was adequately addressed. The calculated floor or emissions limit would need no adjustment. If, on the other hand, the value equal to three times the RDL were greater than the UPL, we would conclude that the calculated floor or emissions limit does not account entirely for measurement variability. If indicated, we substituted the value equal to three times the RDL for the calculated MACT floor UPL emissions level.

Add any RDL content specific to your source category

Table 1: Here is an example of a table in case you need one

	Existing MACT floor (old units)	Existing MACT floor (new units)	New MACT floor
Subcategory 1	HAP 1, HAP 2	HAP 1, HAP 2	HAP 1, HAP 2, X, Y, Z
Subcategory 2	HAP 1	HAP 1	HAP 1, X, Y, Z
Subcategory 3	HAP 1, HAP 2, HAP 3	HAP 1, HAP 2, HAP 3	HAP 1, HAP 2, HAP 3, X, Y, Z
Subcategory 4	None	None	None
Subcategory 5	None	None	X, Y, Z

describe any data that was excluded here and why

Warning: the data for HAP Chemical X contain emissions with a value of zero. These have been removed from the data set prior to analysis, but consider checking for errors or proper significant digit handling.

Tables with the full data sets used for each MACT floor analysis can be found listed in Appendix A.

Existing Source Chemical X for Size subcategory A

The final data set for the Chemical X Existing source MACT floor for Size subcategory A included 63 test runs from 5 sources. The best performer for the new source calculation was D with an average emission of 0.08556 (ppm) from 63 test runs.

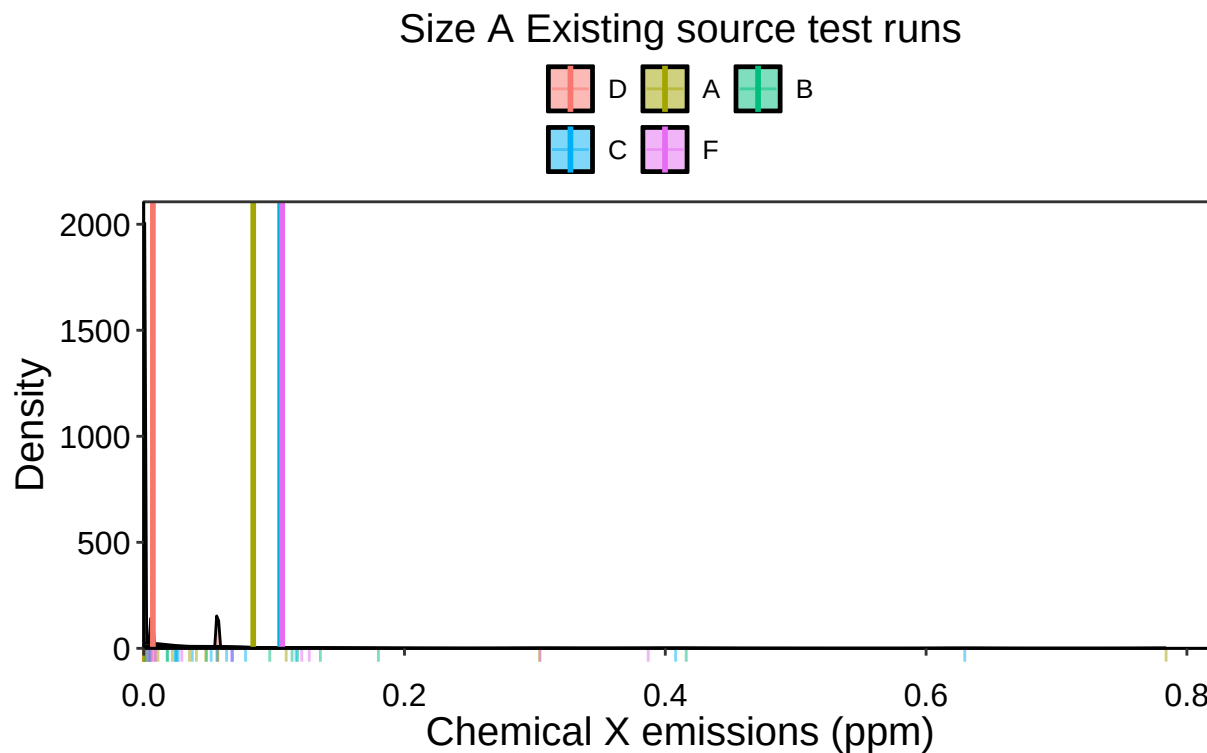


Figure 1: Observation density of emissions by source type. Source averages are indicated by vertical lines.

Existing Source Chemical X for Size subcategory B

The final data set for the Chemical X Existing source MACT floor for Size subcategory B included 64 test runs from 5 sources. The best performer for the new source calculation was D with an average emission of 0.08181 (ppm) from 64 test runs.

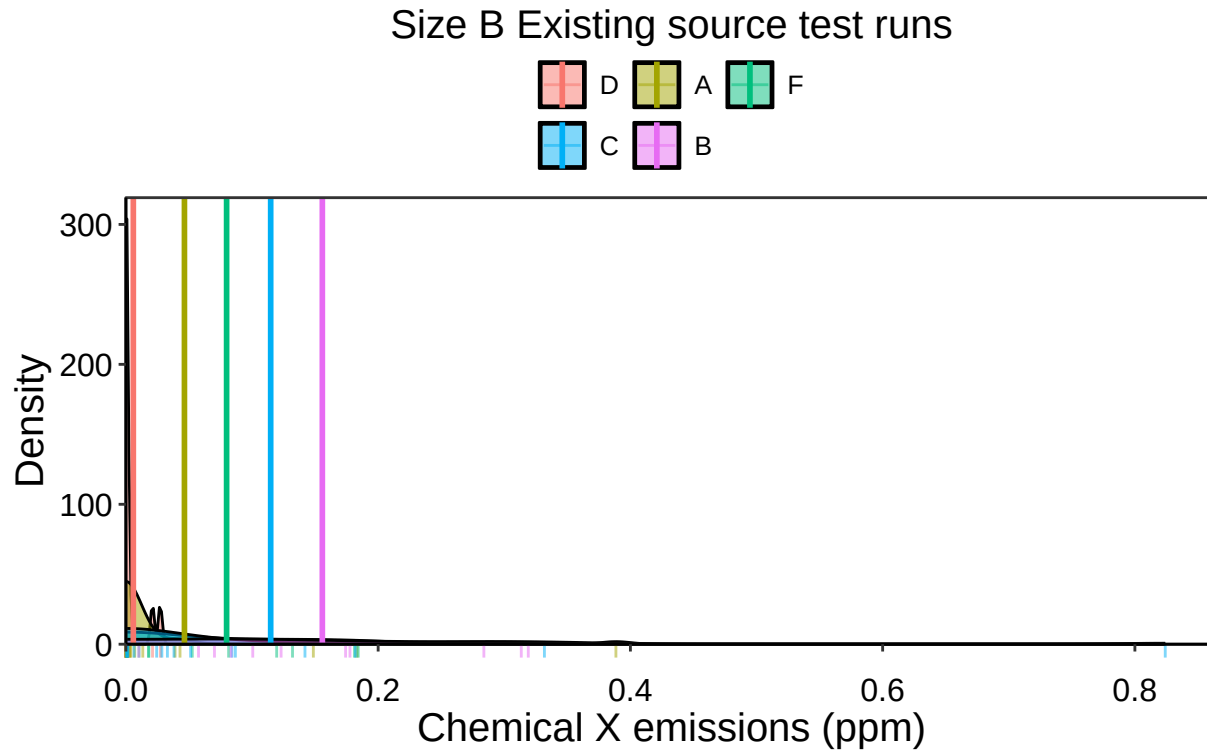


Figure 2: Observation density of emissions by source type. Source averages are indicated by vertical lines.

New Source Chemical X for Size subcategory B

The final data set for the Chemical X New source MACT floor for Size subcategory B included 9 test runs from 1 sources. The best performer for the new source calculation was D with an average emission of 0.005873 (ppm) from 9 test runs.

Existing Source Chemical Y

The final data set for the Chemical Y Existing source MACT floor included 128 test runs from 5 sources. The best performer for the new source calculation was E with an average emission of 3.984 (mg/m³/s) from 128 test runs.

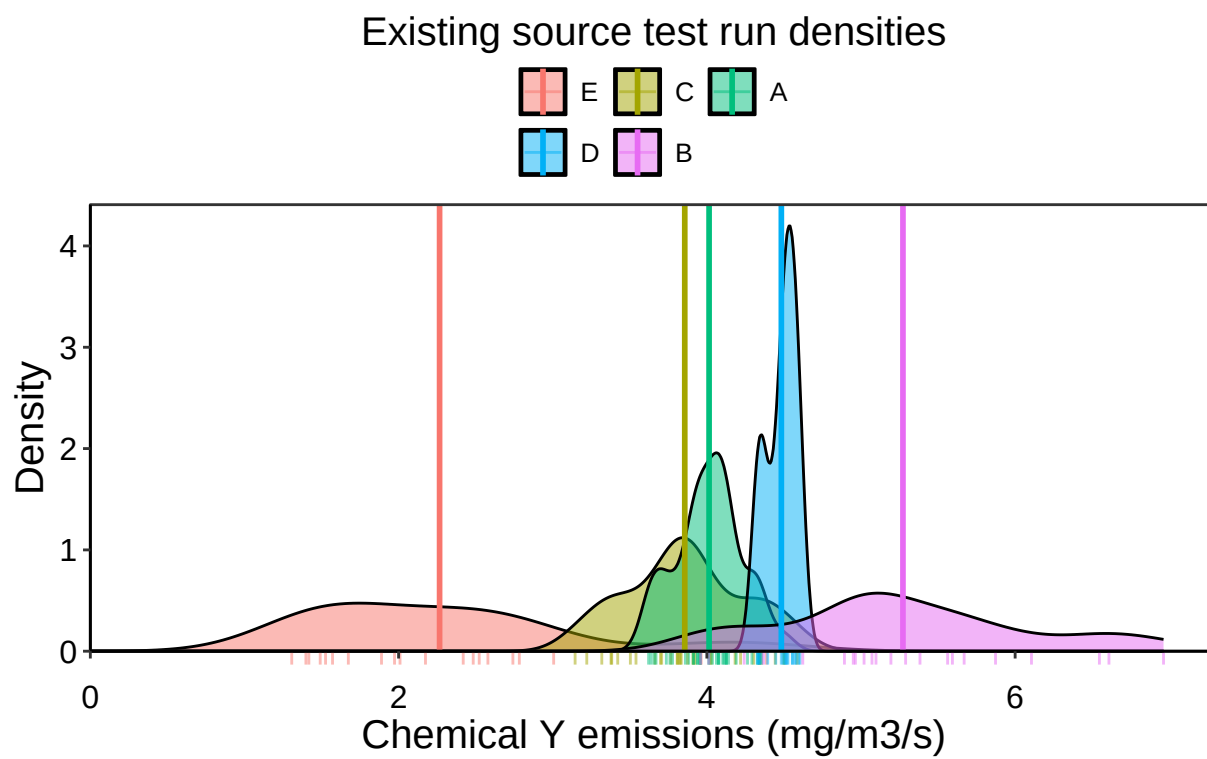


Figure 3: Observation density of emissions by source type. Source averages are indicated by vertical lines.

MACT Floor Analysis Results

Existing Source Chemical X for Size subcategory A

The distribution that best represents the Existing source data set for Chemical X for Size subcategory A is the Skewed distribution, with a corresponding UPL of 0.3007 (ppm).

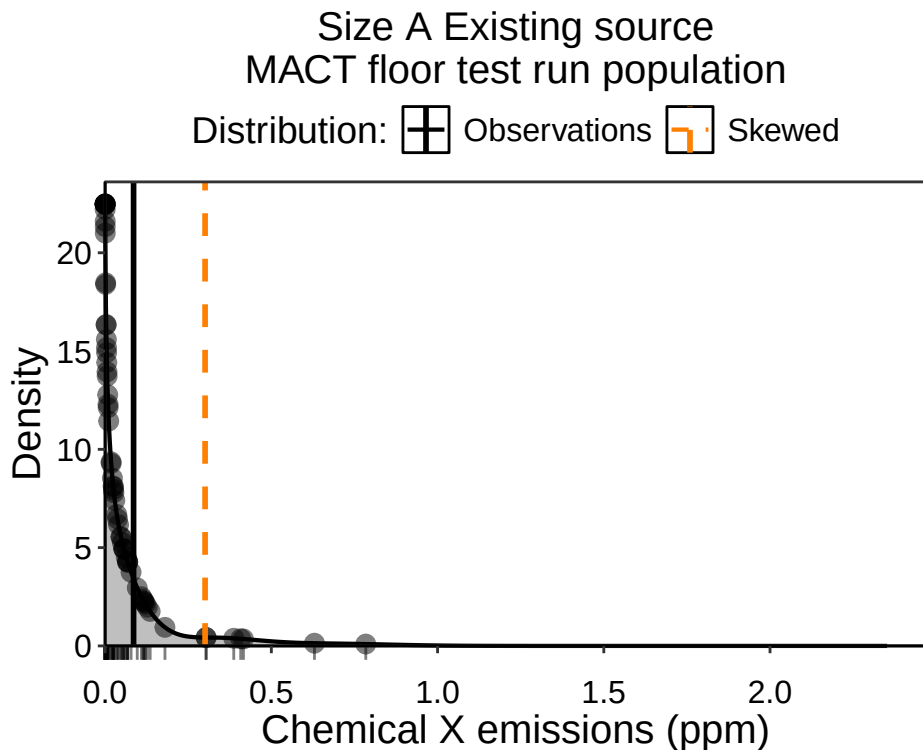


Figure 4: Observation density of emissions for overall population of test runs. The observation data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored orange. The average of the emission test runs is the vertical black line and the UPL result is the vertical orange line.

The density plot above of emission test run data was created using `ggplot2` in R. The density curves were generated using `obs_density()` in `UPLforOAR` version 0.1.0, using a gamma kernel type, bounds of $(0, \infty)$, and a bandwidth of 0.0288. For further details see the R code in Appendix B.

Existing Source Chemical X for Size subcategory B

The distribution that best represents the Existing source data set for Chemical X for Size subcategory B is the Skewed distribution, with a corresponding UPL of 0.2729 (ppm).

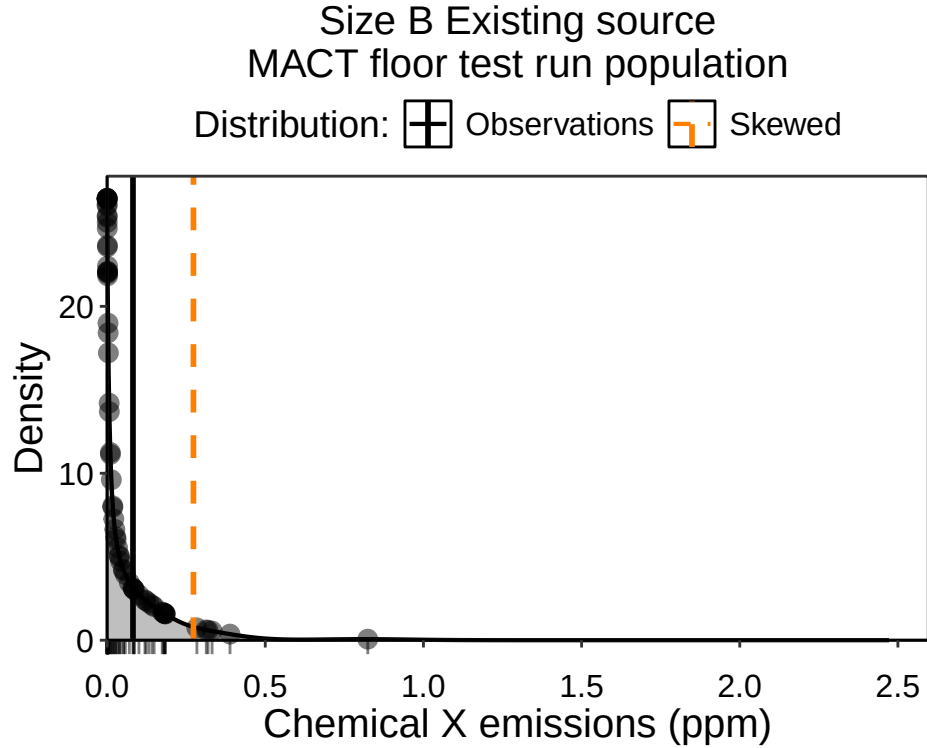


Figure 5: Observation density of emissions for overall population of test runs. The observation data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored orange. The average of the emission test runs is the vertical black line and the UPL result is the vertical orange line.

The density plot above of emission test run data was created using `ggplot2` in R. The density curves were generated using `obs_density()` in `UPLforOAR` version 0.1.0, using a gamma kernel type, bounds of $(0, \infty)$, and a bandwidth of 0.0255. For further details see the R code in Appendix B.

New Source Chemical X for Size subcategory B

The distribution that best represents the New source data set for Chemical X for Size subcategory B is the Skewed distribution, with a corresponding UPL of 0.02461 (ppm).

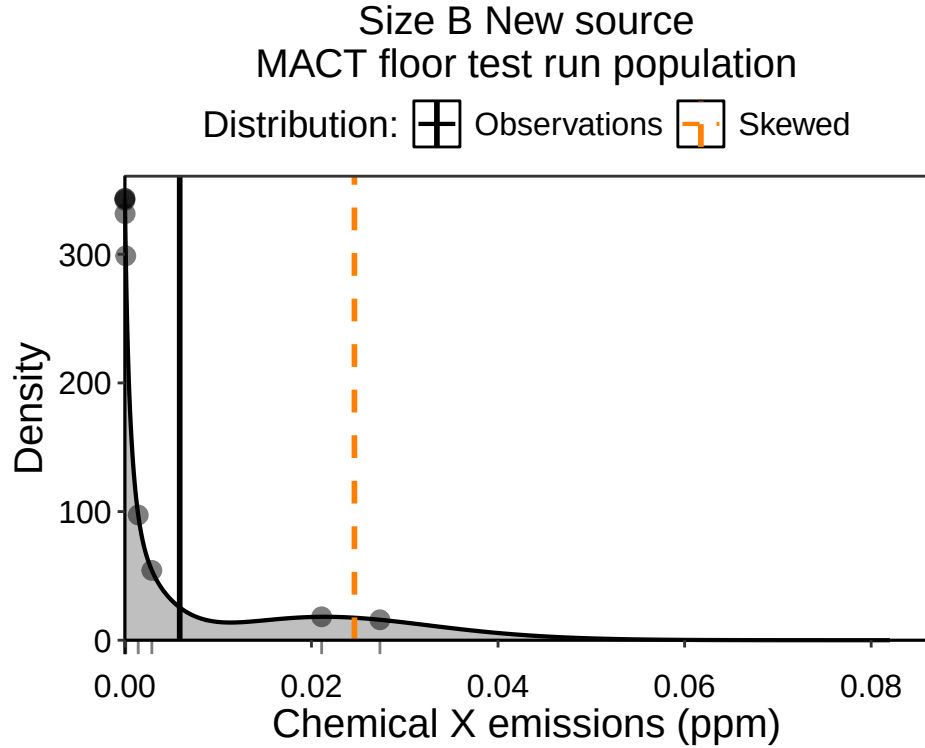


Figure 6: Observation density of emissions for overall population of test runs. The observation data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored orange. The average of the emission test runs is the vertical black line and the UPL result is the vertical orange line.

The density plot above of emission test run data was created using `ggplot2` in R. The density curves were generated using `obs_density()` in `UPLforOAR` version 0.1.0, using a gamma kernel type, bounds of $(0, \infty)$, and a bandwidth of 0.00438. For further details see the R code in Appendix B.

Existing Source Chemical Y

The distribution that best represents the Existing source data set for Chemical Y is the Skewed distribution, with a corresponding UPL of 4.975 (mg/m³/s).

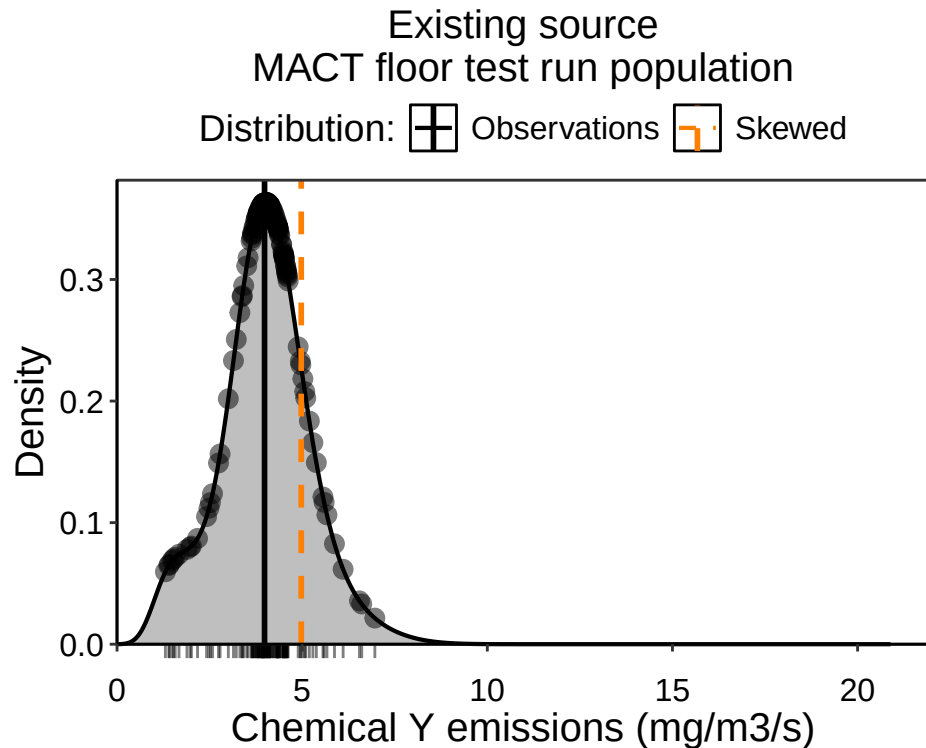


Figure 7: Observation density of emissions for overall population of test runs. The observation data are indicated in black as points and a rug along the axis, with the observation density distribution as a black line. The fitted distribution that is the basis of the UPL estimate is colored orange. The average of the emission test runs is the vertical black line and the UPL result is the vertical orange line.

The density plot above of emission test run data was created using `ggplot2` in R. The density curves were generated using `obs_density()` in `UPLforOAR` version 0.1.0, using a gamma kernel type, bounds of $(0, \infty)$, and a bandwidth of 0.147. For further details see the R code in Appendix B.

UPL Calculation Summary

The table below includes all UPL calculations for Example Industry MACT floor analyses.

Table 2: Upper Predictive Limit (UPL) results for MACT floors in (measurement units)

HAP	SubcategoryName	SubcategoryLevel	Source Type	UPL	Significance	No. FutureRuns
Chemical X	Size	A	Existing	0.30070	0.99	3
Chemical X	Size	B	Existing	0.27290	0.99	3
Chemical X	Size	B	New	0.02461	0.99	3
Chemical Y	NA	NA	Existing	4.97500	0.99	3

References

- Bhaumik, D.K. and Gibbons, R.D. 2004. An Upper Prediction Limit for the Arithmetic Mean of a Lognormal Random Variable. *Technometrics*, 46(2):239-248.
- Dragan Doric, et al. 2007. On measuring skewness and kurtosis. Springer Science + Business Media B. V. 2007.
- Renault O. & O. Scaillet. On the way to recovery: A parametric bias free estimation of recovery rate densities. 2004. *Journal of Banking and Finance*. 28:12 p. 2915-2931.

Appendix A: Example Industry MACT Floor Data Tables

Existing source MACT floor analysis data set for HAP Chemical X for Size subcategory A:

Table 3: Top performing source emissions data used in MACT floor UPL calculations.

chemX	boiler units	log emissions
3.574771e-04	D	-7.9364394
2.900000e-11	D	-24.2520909
6.143717e-03	D	-5.0923253
2.682852e-04	D	-8.2234599
9.278000e-09	D	-18.4956090
1.000000e-12	D	-27.9598118
1.001000e-09	D	-20.7225265
5.668429e-02	D	-2.8702581
5.918990e-07	D	-14.3399294
4.704324e-04	A	-7.6618583
2.199485e-02	A	-3.8169467
3.035256e-01	A	-1.1922894
3.513587e-02	A	-3.3485329
1.088776e-02	A	-4.5201158
7.840528e-01	A	-0.2432790
1.112437e-05	A	-11.4063722
3.059722e-03	A	-5.7894312
1.628269e-03	A	-6.4202378
7.983759e-05	A	-9.4355162
4.758858e-02	A	-3.0451624
5.225030e-06	A	-12.1620501
4.195293e-03	A	-5.4737921
4.044149e-02	A	-3.2078990
1.092620e-01	A	-2.2140070
8.825569e-03	A	-4.7301022
5.687594e-02	A	-2.8668828
1.799690e-01	B	-1.7149704
9.666647e-02	B	-2.3364886
1.580851e-03	B	-6.4497920
1.795918e-02	B	-4.0196539
1.137799e-01	B	-2.1734890
1.179986e-01	B	-2.1370825
1.354828e-01	B	-1.9989107
5.125207e-03	B	-5.2735844
4.160504e-01	B	-0.8769488
4.821298e-02	B	-3.0321270
1.832902e-02	B	-3.9992697
3.753415e-03	C	-5.5850892
4.492662e-03	C	-5.4053099
3.729328e-02	C	-3.2889420
2.639671e-02	C	-3.6345159
1.168308e-01	C	-2.1470284
2.424387e-02	C	-3.7195916
6.296752e-01	C	-0.4625512
2.541257e-02	C	-3.6725115
2.448155e-02	C	-3.7098353

chemX	boiler units	log emissions
5.173075e-02	C	-2.9617029
5.612813e-02	C	-2.8801181
6.808532e-02	C	-2.6869937
4.079142e-01	C	-0.8966985
6.354355e-02	C	-2.7560298
6.780857e-02	C	-2.6910666
7.808603e-02	C	-2.5499441
5.804543e-03	F	-5.1491144
3.040249e-01	F	-1.1906458
7.814457e-03	F	-4.8517798
1.269964e-01	F	-2.0635966
2.918828e-02	F	-3.5339879
3.869393e-01	F	-0.9494875
3.035440e-03	F	-5.7973989
6.753935e-02	F	-2.6950449
1.211970e-01	F	-2.1103376
9.194965e-03	F	-4.6890992

Existing source MACT floor analysis data set for HAP Chemical X for Size subcategory B:

Table 4: Top performing source emissions data used in MACT floor UPL calculations.

chemX	boiler units	log emissions
2.887358e-03	D	-5.8474134
2.831675e-06	D	-12.7746423
2.733793e-02	D	-3.5994800
2.109114e-02	D	-3.8589021
2.000000e-12	D	-26.7626239
9.198713e-05	D	-9.2938619
1.427080e-03	D	-6.5521249
2.240963e-05	D	-10.7060197
6.396610e-07	D	-14.2623270
3.134272e-04	A	-8.0679435
3.471207e-06	A	-12.5710082
1.337857e-02	A	-4.3141009
3.973467e-03	A	-5.5281163
1.446685e-03	A	-6.5384805
4.077026e-04	A	-7.8049725
1.832067e-01	A	-1.6971403
1.486117e-01	A	-1.9064185
1.476972e-03	A	-6.5177612
3.216389e-03	A	-5.7394960
3.885034e-01	A	-0.9454535
1.299787e-03	A	-6.6455549
1.219803e-04	A	-9.0116514
9.515081e-06	A	-11.5626325
3.897209e-02	A	-3.2449096
6.973811e-03	A	-4.9655934
4.289383e-02	A	-3.1490273
5.140600e-04	A	-7.5731706

chemX	boiler units	log emissions
1.320970e-01	F	-2.0242186
1.841823e-01	F	-1.6918294
1.818999e-02	F	-4.0068836
8.780006e-04	F	-7.0378632
5.260468e-02	F	-2.9449502
1.812094e-01	F	-1.7081020
1.786859e-02	F	-4.0247107
8.144771e-02	F	-2.5077940
1.195417e-01	F	-2.1240901
1.040103e-02	F	-4.5658501
8.676420e-02	C	-2.4445612
5.128411e-02	C	-2.9703744
2.850093e-02	C	-3.5578186
1.820212e-01	C	-1.7036323
1.532580e-03	C	-6.4808027
1.419820e-01	C	-1.9520552
1.427751e-03	C	-6.5516548
3.290092e-02	C	-3.4142547
2.439587e-02	C	-3.7133415
8.403949e-02	C	-2.4764685
2.848149e-04	C	-8.1636710
3.318240e-01	C	-1.1031506
8.484343e-04	C	-7.0721179
3.812556e-02	C	-3.2668703
6.449463e-03	C	-5.0437584
8.240595e-01	C	-0.1935126
7.020865e-02	B	-2.6562838
1.230280e-01	B	-2.0953436
5.757417e-02	B	-2.8546813
1.005689e-01	B	-2.2969123
3.190250e-01	B	-1.1424859
1.020272e-02	B	-4.5851009
1.776374e-01	B	-1.7280107
2.838214e-01	B	-1.2594100
8.337441e-02	B	-2.4844139
3.134784e-01	B	-1.1600247
1.741825e-01	B	-1.7476515

New source MACT floor analysis data set for HAP Chemical X for Size subcategory B:

Table 5: Top performing source emissions data used in MACT floor UPL calculations.

chemX	boiler units	log emissions
2.887358e-03	D	-5.847413
2.831675e-06	D	-12.774642
2.733793e-02	D	-3.599480
2.109114e-02	D	-3.858902
2.000000e-12	D	-26.762624
9.198713e-05	D	-9.293862
1.427080e-03	D	-6.552125

chemX	boiler units	log emissions
2.240963e-05	D	-10.706020
6.396610e-07	D	-14.262327

Existing source MACT floor analysis data set for HAP Chemical Y :

Table 6: Top performing source emissions data used in MACT floor UPL calculations.

chemY	boiler units	log emissions
2.174424	E	0.7767640
1.673932	E	0.5151754
1.889162	E	0.6361334
2.783342	E	1.0236523
2.419206	E	0.8834394
3.006086	E	1.1006388
1.306758	E	0.2675496
1.491378	E	0.3997004
1.571521	E	0.4520438
2.009255	E	0.6977641
1.398739	E	0.3355712
1.526092	E	0.4227104
1.417813	E	0.3491157
2.523017	E	0.9254552
2.484151	E	0.9099311
2.741503	E	1.0085063
3.963096	E	1.3770257
1.973820	E	0.6799708
2.579510	E	0.9475996
4.386490	E	1.4785294
3.540232	C	1.2641922
3.382720	C	1.2186803
3.421676	C	1.2301305
4.344752	C	1.4689686
4.586216	C	1.5230554
4.303682	C	1.4594710
3.663421	C	1.2983974
3.145160	C	1.1458648
3.911287	C	1.3638665
4.356717	C	1.4717187
4.218697	C	1.4395264
3.957258	C	1.3755515
3.911109	C	1.3638210
3.378192	C	1.2173407
4.021903	C	1.3917551
3.827899	C	1.3423160
3.833986	C	1.3439050
3.699019	C	1.3080677
3.220992	C	1.1696895
3.503628	C	1.2537991
4.183033	C	1.4310365
3.819464	C	1.3401101

chemY	boiler units	log emissions
3.807323	C	1.3369264
3.706842	C	1.3101802
4.500295	C	1.5041430
3.809243	C	1.3374304
3.924598	C	1.3672639
3.874610	C	1.3544451
3.318383	C	1.1994777
3.766767	C	1.3262170
4.443461	C	1.4914336
4.029063	C	1.3935338
4.292470	A	1.4568624
4.140423	A	1.4207980
4.501422	A	1.5043934
3.621995	A	1.2870249
3.882037	A	1.3563601
3.657100	A	1.2966704
3.915800	A	1.3650198
4.062028	A	1.4016824
4.340514	A	1.4679928
3.911323	A	1.3638757
3.760281	A	1.3244936
3.938524	A	1.3708060
4.124211	A	1.4168747
4.036380	A	1.3953484
4.263680	A	1.4501327
3.940218	A	1.3712361
3.670356	A	1.3002886
4.190300	A	1.4327723
3.777375	A	1.3290294
4.123393	A	1.4166764
4.102699	A	1.4116450
4.073892	A	1.4045988
4.344779	A	1.4689749
3.958771	A	1.3759336
3.737161	A	1.3183262
4.012695	A	1.3894630
3.953605	A	1.3746277
4.127012	A	1.4175537
4.326929	A	1.4648580
3.638334	A	1.2915260
3.957223	A	1.3755425
4.014073	A	1.3898065
4.109942	A	1.4134088
3.860635	A	1.3508318
4.078113	A	1.4056344
4.076282	A	1.4051854
4.354797	D	1.4712780
4.507734	D	1.5057946
4.555990	D	1.5164427
4.492505	D	1.5024105
4.578270	D	1.5213211
4.482733	D	1.5002329

chemY	boiler units	log emissions
4.394871	D	1.4804381
4.572945	D	1.5201574
4.526267	D	1.5098974
4.554366	D	1.5160863
4.604106	D	1.5269485
4.519751	D	1.5084568
4.337042	D	1.4671926
4.512104	D	1.5067636
4.336258	D	1.4670117
4.446896	D	1.4922063
4.595368	D	1.5250488
4.327935	D	1.4650905
5.289483	B	1.6657205
5.194230	B	1.6475484
6.546733	B	1.8789661
5.872839	B	1.7703382
6.962961	B	1.9406049
5.071315	B	1.6236002
4.242131	B	1.4450658
4.963653	B	1.6021419
5.382787	B	1.6832063
5.097371	B	1.6287249
4.949946	B	1.5993767
3.959365	B	1.3760836
4.622315	B	1.5308957
4.368199	B	1.4743507
5.562062	B	1.7159690
5.593076	B	1.7215294
6.608701	B	1.8883871
5.019913	B	1.6134126
6.106131	B	1.8092933
4.892633	B	1.5877306
4.012346	B	1.3893762
5.669356	B	1.7350756

Appendix B: R Function Code

Disclaimer

The United States Environmental Protection Agency (EPA) GitHub project code is provided on an “as is” basis and the user assumes responsibility for its use. EPA has relinquished control of the information and no longer has responsibility to protect the integrity, confidentiality, or availability of the information. Any reference to specific commercial products, processes, or services by service mark, trademark, manufacturer, or otherwise, does not constitute or imply their endorsement, recommendation or favoring by EPA. The EPA seal and logo shall not be used in any manner to imply endorsement of any commercial product or activity by EPA or the United States Government.

MACT_existing()

```
#> function (data, emissions, sources, CAA_section = 112, national_N = NA)
#> {
#>   data_temp = tibble::tibble(emissions = data[[emissions]],
#>     sources = data[[sources]])
#>   if (!is.numeric(data_temp$emissions)) {
#>     stop("Emissions must be numeric vector")
#>   }
#>   if ((!is.character(data_temp$sources)) & (!is.factor(data_temp$sources))) {
#>     stop("Sources must be a character or factor vector")
#>   }
#>   if (any(data_temp$emissions < 0)) {
#>     warning("emissions data contain negative values")
#>   }
#>   if (any(data_temp$emissions == 0)) {
#>     warning("emissions data contain zero values and have been removed")
#>     data_temp = subset(data_temp, data_temp$emissions !=
#>       0)
#>   }
#>   dat_means = dplyr::summarize(data_temp, means = mean(emissions),
#>     .by = "sources")
#>   n_sources = length(unique(data_temp$sources))
#>   if (CAA_section == 129) {
#>     if (is.na(national_N)) {
#>       stop("Must provide total number of sources as national_N")
#>     }
#>     n_topsources = ceiling(0.12 * national_N)
#>     if (n_sources < n_topsources) {
#>       n_topsources = n_sources
#>     }
#>     top_list = dat_means[order(dat_means$means, decreasing = F),
#>       ]
#>     top_list = top_list$sources[1:n_topsources]
#>     dat_top = subset(data_temp, data_temp$sources %in% top_list)
#>   }
#>   else if (CAA_section == 112) {
#>     if (n_sources >= 30) {
#>       n_topsources = ceiling(0.12 * n_sources)
#>       top_list = dat_means[order(dat_means$means, decreasing = F),
#>         ]
#>       top_list = top_list$sources[1:n_topsources]
#>       dat_top = subset(data_temp, data_temp$sources %in%
```

```

#>         top_list)
#>     }
#>     else if (n_sources < 30) {
#>         n_topsources = 5
#>         top_list = dat_means[order(dat_means$means, decreasing = F),
#>                                ]
#>         top_list = top_list$sources[1:n_topsources]
#>         dat_top = subset(data_temp, data_temp$sources %in%
#>                           top_list)
#>     }
#> }
#> dat_topmeans = dplyr::summarize(dat_top, means = mean(emissions),
#>                                .by = "sources", counts = dplyr::n())
#> dat_topmeans$sources = as.factor(dat_topmeans$sources)
#> dat_topmeans$sources = forcats::fct_reorder(dat_topmeans$sources,
#>        dat_topmeans$means, .desc = FALSE)
#> dat_top$sources = factor(dat_top$sources, levels = levels(dat_topmeans$sources))
#> dat_topmeans = dplyr::arrange(dat_topmeans, means)
#> dat_top = dplyr::arrange(dat_top, sources)
#> dat_top$sources = droplevels(dat_top$sources)
#> if (!is.character(emissions)) {
#>     emissions = colnames(data)[emissions]
#> }
#> if (!is.character(sources)) {
#>     sources = colnames(data)[sources]
#> }
#> names(dat_top) = c(emissions, sources)
#> return(dat_top)
#> }
#> <bytecode: 0x000001df537acd30>
#> <environment: namespace:UPLforOAR>

```

MACT_new()

```

#> function (data, emissions, sources)
#> {
#>     data_temp = tibble::tibble(emissions = data[[emissions]],
#>                                sources = data[[sources]])
#>     if (!is.numeric(data_temp$emissions)) {
#>         stop("Emissions must be numeric vector")
#>     }
#>     if ((!is.character(data_temp$sources)) & (!is.factor(data_temp$sources))) {
#>         stop("Sources must be a character or factor vector")
#>     }
#>     if (any(data_temp$emissions < 0)) {
#>         warning("emissions data contain negative values")
#>     }
#>     if (any(data_temp$emissions == 0)) {
#>         warning("emissions data contain zero values and have been removed")
#>         data_temp = subset(data_temp, data_temp$emissions !=
#>                             0)
#>     }
#>     dat_means = dplyr::summarize(data_temp, means = mean(emissions),
#>                                .by = "sources")

```

```

#>   top_list = dat_means[order(dat_means$means, decreasing = F),
#>   ]
#>   top_source = top_list$sources[1]
#>   dat_top = subset(data_temp, data_temp$sources == top_source)
#>   dat_top$sources = as.factor(dat_top$sources)
#>   dat_top$sources = droplevels(dat_top$sources)
#>   if (!is.character(emissions)) {
#>     emissions = colnames(data)[emissions]
#>   }
#>   if (!is.character(sources)) {
#>     sources = colnames(data)[sources]
#>   }
#>   names(dat_top) = c(emissions, sources)
#>   return(dat_top)
#> }
#> <bytecode: 0x000001df5e453f98>
#> <environment: namespace:UPLforOAR>

```

distribution_type()

```

#> function (data, emissions)
#> {
#>   data_temp = tibble::tibble(emissions = data[[emissions]])
#>   data_temp$ln_emiss = log(data_temp$emissions)
#>   data_temp$ln_emiss = replace(data_temp$ln_emiss, !is.finite(data_temp$ln_emiss),
#>   NA)
#>   sigma = stats::sd(data_temp$emissions)
#>   mean_ln = mean(data_temp$ln_emiss, na.rm = TRUE)
#>   sigma_ln = stats::sd(data_temp$ln_emiss, na.rm = TRUE)
#>   if ((sigma == 0) | (sigma_ln == 0)) {
#>     stop("Cannot perform UPL calculation on data with 0 variance")
#>   }
#>   n = length(data_temp$emissions)
#>   if (n < 3) {
#>     stop("Need at least 3 observations for UPL calculation")
#>   }
#>   emission_mean = mean(data_temp$emissions)
#>   S = n/((n - 1) * (n - 2)) * sum(((data_temp$emissions - emission_mean)/sigma)^3)
#>   S_ln = n/((n - 1) * (n - 2)) * sum(((data_temp$ln_emiss -
#>   mean_ln)/sigma_ln)^3, na.rm = T)
#>   SES = sqrt((6 * n * (n - 1))/((n - 2) * (n + 1) * (n + 3)))
#>   S_SES = abs(S/SES)
#>   S_SES_ln = abs(S_ln/SES)
#>   if (n == 3) {
#>     K = sum((data_temp$emissions - emission_mean)^4)/((n -
#>     1) * (sigma)^4) - 3
#>     K_ln = sum((data_temp$ln_emiss - mean_ln)^4, na.rm = T)/((n -
#>     1) * (sigma_ln)^4) - 3
#>     SEK = sqrt(24/n)
#>     if (S_SES < S_SES_ln) {
#>       distr_choice = "Normal"
#>     }
#>   }
#>   else {
#>     distr_choice = "Lognormal"
#>   }
#> }

```

```

#>     }
#>   }
#>   else if (n > 3) {
#>     K = (n * (n + 1))/((n - 1) * (n - 2) * (n - 3)) * sum(((data_temp$emissions -
#>       emission_mean)/sigma)^4) - (3 * (n - 1)^2)/((n -
#>       2) * (n - 3))
#>     K_ln = (n * (n + 1))/((n - 1) * (n - 2) * (n - 3)) *
#>       sum(((data_temp$ln_emiss - mean_ln)/sigma_ln)^4,
#>       na.rm = T) - (3 * (n - 1)^2)/((n - 2) * (n -
#>       3))
#>     SEK = sqrt(24 * n * (n^2 - 1)/((n - 2) * (n + 3) * (n -
#>       3) * (n + 5)))
#>     S_SEK = abs(K/SEK)
#>     S_SEK_ln = abs(K_ln/SEK)
#>     norm_zscore = stats::qnorm(0.975)
#>     if (S_SES > norm_zscore) {
#>       raw_distr1 = "Non-normal"
#>     }
#>     else {
#>       raw_distr1 = "Normal"
#>     }
#>     if (S_SEK > norm_zscore) {
#>       raw_distr2 = "Non-normal"
#>     }
#>     else {
#>       raw_distr2 = "Normal"
#>     }
#>     if (S_SES_ln > norm_zscore) {
#>       ln_distr1 = "Non-normal"
#>     }
#>     else {
#>       ln_distr1 = "Normal"
#>     }
#>     if (S_SEK_ln > norm_zscore) {
#>       ln_distr2 = "Non-normal"
#>     }
#>     else {
#>       ln_distr2 = "Normal"
#>     }
#>     if ((raw_distr1 == "Normal") & (raw_distr2 == "Normal")) {
#>       raw_distr = "Normal"
#>     }
#>     else {
#>       raw_distr = "Non-normal"
#>     }
#>     if ((ln_distr1 == "Normal") & (ln_distr2 == "Normal")) {
#>       ln_distr = "Normal"
#>     }
#>     else {
#>       ln_distr = "Non-normal"
#>     }
#>     if ((ln_distr == "Normal") & (raw_distr == "Normal")) {
#>       if (S_SES < S_SES_ln) {
#>         distr_choice = "Normal"

```



```

#>     }
#>     else {
#>         distr_choice = "Lognormal"
#>     }
#> }
#> else if (raw_distr == "Normal") {
#>     distr_choice = "Normal"
#> }
#> else if (ln_distr == "Normal") {
#>     distr_choice = "Lognormal"
#> }
#> else {
#>     distr_choice = "Skewed"
#> }
#> }
#> return(distr_choice)
#> }
#> <bytecode: 0x000001df4b054178>
#> <environment: namespace:UPLforOAR>

```

Normal_UPL()

```

#> function (data, emissions, future_runs = 3, significance = 0.99)
#> {
#>     future_runs = as.integer(future_runs)
#>     if (!is.integer(future_runs)) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     if (future_runs < 1) {
#>         stop("future_runs must be a positive integer")
#>     }
#>     if (significance >= 1) {
#>         stop("significance must be greater than 0 and less than 1")
#>     }
#>     if (significance <= 0) {
#>         stop("significance must be greater than 0 and less than 1")
#>     }
#>     data_temp = tibble::tibble(emissions = data[[emissions]])
#>     n = length(data_temp$emissions)
#>     if (n < 3) {
#>         stop("Need at least 3 observations for UPL calculation")
#>     }
#>     df = n - 1
#>     tscore = stats::qt(significance, df)
#>     emission_mean = mean(data_temp$emissions)
#>     var.s = sum((data_temp$emissions - emission_mean)^2) * (1/(n -
#>         1))
#>     if (var.s == 0) {
#>         stop("Cannot perform UPL calculation on data with 0 variance")
#>     }
#>     UPL_normal = emission_mean + tscore * sqrt(var.s * (1/n +
#>         1/future_runs))
#>     return(UPL_normal)
#> }

```

```
#> <bytecode: 0x000001df53ab2710>
#> <environment: namespace:UPLforOAR>
```

Lognormal_UPL()

```
#> function (data, emissions, future_runs = 3, significance = 0.99)
#> {
#>   future_runs = as.integer(future_runs)
#>   if (!is.integer(future_runs)) {
#>     stop("future_runs must be a positive integer")
#>   }
#>   if (future_runs < 1) {
#>     stop("future_runs must be a positive integer")
#>   }
#>   if (significance >= 1) {
#>     stop("significance must be greater than 0 and less than 1")
#>   }
#>   if (significance <= 0) {
#>     stop("significance must be greater than 0 and less than 1")
#>   }
#>   data_temp = tibble::tibble(emissions = data[[emissions]])
#>   data_temp$ln_emiss = log(data_temp$emissions)
#>   data_temp$ln_emiss = replace(data_temp$ln_emiss, !is.finite(data_temp$ln_emiss),
#>     NA)
#>   n = length(data_temp$emissions)
#>   if (n < 3) {
#>     stop("Need at least 3 observations for UPL calculation")
#>   }
#>   mean_log = mean(data_temp$ln_emiss, na.rm = TRUE)
#>   sd_log = stats::sd(data_temp$ln_emiss, na.rm = TRUE)
#>   if (sd_log == 0) {
#>     stop("Cannot perform UPL calculation on data with 0 variance")
#>   }
#>   beta2 = (exp(4 * sd_log^2) + 2 * exp(3 * sd_log^2) + 3 *
#>     exp(2 * sd_log^2) - 3)/(future_runs * (exp(sd_log^2) -
#>     1)^2) + 3 * (1 - 1/future_runs)
#>   beta1 = sqrt(exp(sd_log^2) - 1) * (exp(sd_log^2) + 2)/sqrt(future_runs)
#>   zvals = seq(-5, 5, by = 0.0101)
#>   Fgzs = abs((1 - (beta1/6) * (3 * zvals - zvals^3) + (beta2 -
#>     3) * (3 - 6 * zvals^2 + zvals^4)/24) * stats::dnorm(zvals))
#>   Fg_cdf = cumsum(Fgzs/sum(Fgzs))
#>   zscore1 = zvals[Fg_cdf > significance][1]
#>   mean_convert = exp(mean_log + (sd_log^2)/2)
#>   part2 = sqrt(future_runs * exp(2 * mean_log + sd_log^2) *
#>     (exp(sd_log^2) - 1) + future_runs^2 * exp(2 * mean_log +
#>     sd_log^2) * (((sd_log^2)/n) + (sd_log^4)/(2 * (n - 1))))
#>   UPL_lognormal = mean_convert + (zscore1/future_runs) * part2
#>   return(UPL_lognormal)
#> }
```

```
#> <bytecode: 0x000001df5049bd60>
#> <environment: namespace:UPLforOAR>
```

Skewed_UPL()

```
#> function (data, emissions, future_runs = 3, significance = 0.99)
```

```

#> {
#>   data_temp = tibble::tibble(emissions = data[[emissions]])
#>   n = length(data_temp$emissions)
#>   if (n < 3) {
#>     stop("Need at least 3 observations for UPL calculation")
#>   }
#>   if (n > 341) {
#>     warning("slower calculations due to requirement for high precision floating point")
#>   }
#>   future_runs = as.integer(future_runs)
#>   emission_mean = mean(data_temp$emissions, na.rm = T)
#>   sigma = stats::sd(data_temp$emissions, na.rm = T)
#>   if (sigma == 0) {
#>     stop("Cannot perform UPL calculation on data with 0 variance")
#>   }
#>   if (!is.integer(future_runs)) {
#>     stop("future_runs must be a positive integer")
#>   }
#>   if (future_runs < 1) {
#>     stop("future_runs must be a positive integer")
#>   }
#>   if (significance >= 1) {
#>     stop("significance must be greater than 0 and less than 1")
#>   }
#>   if (significance <= 0) {
#>     stop("significance must be greater than 0 and less than 1")
#>   }
#>   if (n <= 3) {
#>     Skewed_UPL = NA
#>     warning("data must have more than 3 observations for skew UPL method")
#>   }
#>   else {
#>     K = (n * (n + 1))/((n - 1) * (n - 2) * (n - 3)) * sum(((data_temp$emissions -
#>       emission_mean)/sigma)^4) - (3 * (n - 1)^2)/((n -
#>       2) * (n - 3))
#>     S = n/((n - 1) * (n - 2)) * sum(((data_temp$emissions -
#>       emission_mean)/sigma)^3)
#>     df = n - 1
#>     var.s = sum((data_temp$emissions - mean(data_temp$emissions))^2) *
#>       (1/(n - 1))
#>     tscore = stats::qt(significance, df)
#>     u0 = 1/(1 + (tscore^2/(n - 1)))
#>     b = c(0.5, 0.5, 0.5, 0.5, 1, 1)
#>     w = b * (u0/(1 - u0))
#>     a = c((n - 1)/2, (n + 1)/2, (n + 3)/2, (n + 5)/2, (n -
#>       1)/2, (n + 1)/2)
#>     I_term = c()
#>     for (i in 1:6) {
#>       c1 = stats::pgamma(w[i], shape = a[i], rate = 1)
#>       c4 = (a[i] - 1 - w[i])/(2 * b[i])
#>       c5 = (a[i]^3/2 - 5 * a[i]^2/3 + 3 * a[i]/2 - 1/3)
#>       c6 = w[i] * (3 * a[i]^2/2 - 11 * a[i]/6 + 1/3)
#>       c7 = w[i]^2 * (3 * a[i]/2 - 1/6)
#>       if (n > 341) {

```

```

#>         c2 = Rmpfr::igamma(a[i], 0)
#>         c3 = ((exp(-w[i]) * w[i]^Rmpfr::mpfr(a[i], 128))/Rmpfr::igamma(a[i],
#>         0))
#>         I_term[i] = Rmpfr::asNumeric(c1/c2 + c3 * (c4 +
#>         (1/(2 * b[i])^2) * (c5 - c6 + c7 - w[i]^3/2)))
#>     }
#>     else {
#>         c2 = gamma(a[i])
#>         c3 = ((exp(-w[i]) * w[i]^a[i])/gamma(a[i]))
#>         I_term[i] = (c1/c2 + c3 * (c4 + (1/(2 * b[i])^2) *
#>         (c5 - c6 + c7 - w[i]^3/2)))
#>     }
#> }
#> coeff1 = (2 * n - 1) * I_term[[5]]/(6 * sqrt(2 * n *
#>     pi)) - (n - 1) * I_term[[6]]/(3 * sqrt(2 * n * pi))
#> coeff2 = (n - 1) * I_term[1]/24 - (n - 1) * (n + 2) *
#>     I_term[2]/(12 * n) + (n + 4) * (n - 1) * I_term[3]/(24 *
#>     n)
#> coeff3 = (n - 1) * (2 * n + 5) * I_term[1]/72 - (n -
#>     1) * (2 * n^2 + 5 * n + 8) * I_term[2]/(24 * n) +
#>     (n - 1) * (2 * n^2 + 5 * n + 12) * I_term[3]/(24 *
#>     n) - (n - 1) * (2 * n^2 + 5 * n + 12) * I_term[4]/(72 *
#>     n)
#> current_prob = 1 - (I_term[1]/2 + S * coeff1 - K * coeff2 +
#>     S^2 * coeff3)
#> if (abs(current_prob - significance) < 1e-04) {
#>     Skewed_UPL = emission_mean + tscore * sqrt(var.s *
#>     (1/n + 1/future_runs))
#> }
#> else {
#>     tstat_list = seq(from = (tscore - 1), to = (tscore +
#>     1), by = 1e-04)
#>     new_prob = c()
#>     for (t in 1:length(tstat_list)) {
#>         u0 = 1/(1 + (tstat_list[t]^2/(n - 1)))
#>         b = c(0.5, 0.5, 0.5, 0.5, 1, 1)
#>         w = b * (u0/(1 - u0))
#>         a = c((n - 1)/2, (n + 1)/2, (n + 3)/2, (n + 5)/2,
#>         (n - 1)/2, (n + 1)/2)
#>         I_term = c()
#>         for (i in 1:6) {
#>             c1 = stats::pgamma(w[i], shape = a[i], rate = 1)
#>             c4 = (a[i] - 1 - w[i])/(2 * b[i])
#>             c5 = (a[i]^3/2 - 5 * a[i]^2/3 + 3 * a[i]/2 -
#>             1/3)
#>             c6 = w[i] * (3 * a[i]^2/2 - 11 * a[i]/6 + 1/3)
#>             c7 = w[i]^2 * (3 * a[i]/2 - 1/6)
#>             if (n > 341) {
#>                 c2 = Rmpfr::igamma(a[i], 0)
#>                 c3 = ((exp(-w[i]) * w[i]^Rmpfr::mpfr(a[i],
#>                 128))/Rmpfr::igamma(a[i], 0))
#>                 I_term[i] = Rmpfr::asNumeric(c1/c2 + c3 *
#>                 (c4 + (1/(2 * b[i])^2) * (c5 - c6 + c7 -
#>                 w[i]^3/2)))

```

```

#>     }
#>     else {
#>         c2 = gamma(a[i])
#>         c3 = ((exp(-w[i]) * w[i]^a[i])/gamma(a[i]))
#>         I_term[i] = (c1/c2 + c3 * (c4 + (1/(2 * b[i])^2) *
#>             (c5 - c6 + c7 - w[i]^3/2)))
#>     }
#> }
#> coeff1 = (2 * n - 1) * I_term[5]/(6 * sqrt(2 *
#>     n * pi)) - (n - 1) * I_term[6]/(3 * sqrt(2 *
#>     n * pi))
#> coeff2 = (n - 1) * I_term[1]/24 - (n - 1) * (n +
#>     2) * I_term[2]/(12 * n) + (n + 4) * (n - 1) *
#>     I_term[3]/(24 * n)
#> coeff3 = (n - 1) * (2 * n + 5) * I_term[1]/72 -
#>     (n - 1) * (2 * n^2 + 5 * n + 8) * I_term[2]/(24 *
#>     n) + (n - 1) * (2 * n^2 + 5 * n + 12) * I_term[3]/(24 *
#>     n) - (n - 1) * (2 * n^2 + 5 * n + 12) * I_term[4]/(72 *
#>     n)
#> new_prob[t] = 1 - (I_term[1]/2 + S * coeff1 -
#>     K * coeff2 + S^2 * coeff3)
#> }
#> good_tscore = which(abs(new_prob - significance) <
#>     1e-04)
#> if (length(good_tscore) > 0) {
#>     new_tscore = tstat_list[good_tscore[which.min(abs(good_tscore -
#>         which(tstat_list == tscore))]]]
#>     Skewed_UPL = mean(data_temp$emissions) + new_tscore *
#>         sqrt(var.s * (1/n + 1/future_runs))
#> }
#> else if (dplyr::between(significance, min(new_prob),
#>     max(new_prob))) {
#>     closest_t = tstat_list[which.min(abs(new_prob -
#>         significance))]
#>     reset_t1 = tstat_list[which.min(abs(new_prob -
#>         significance)) - 1]
#>     reset_t2 = tstat_list[which.min(abs(new_prob -
#>         significance)) + 1]
#>     tstat_list = seq(from = reset_t1, to = reset_t2,
#>         length.out = 5000)
#>     new_prob = c()
#>     for (t in 1:length(tstat_list)) {
#>         u0 = 1/(1 + (tstat_list[t]^2/(n - 1)))
#>         b = c(0.5, 0.5, 0.5, 0.5, 1, 1)
#>         w = b * (u0/(1 - u0))
#>         a = c((n - 1)/2, (n + 1)/2, (n + 3)/2, (n +
#>             5)/2, (n - 1)/2, (n + 1)/2)
#>         I_term = c()
#>         for (i in 1:6) {
#>             c1 = stats::pgamma(w[i], shape = a[i], rate = 1)
#>             c4 = (a[i] - 1 - w[i])/(2 * b[i])
#>             c5 = (a[i]^3/2 - 5 * a[i]^2/3 + 3 * a[i]/2 -
#>                 1/3)
#>             c6 = w[i] * (3 * a[i]^2/2 - 11 * a[i]/6 +

```

```

#>          1/3)
#>      c7 = w[i]^2 * (3 * a[i]/2 - 1/6)
#>      if (n > 341) {
#>          c2 = Rmpfr::igamma(a[i], 0)
#>          c3 = ((exp(-w[i]) * w[i]^Rmpfr::mpfr(a[i],
#>              128))/Rmpfr::igamma(a[i], 0))
#>          I_term[i] = Rmpfr::asNumeric(c1/c2 + c3 *
#>              (c4 + (1/(2 * b[i]))^2) * (c5 - c6 + c7 -
#>              w[i]^3/2)))
#>      }
#>      else {
#>          c2 = gamma(a[i])
#>          c3 = ((exp(-w[i]) * w[i]^a[i])/gamma(a[i]))
#>          I_term[i] = (c1/c2 + c3 * (c4 + (1/(2 *
#>              b[i]))^2) * (c5 - c6 + c7 - w[i]^3/2)))
#>      }
#>  }
#>      coeff1 = (2 * n - 1) * I_term[5]/(6 * sqrt(2 *
#>          n * pi)) - (n - 1) * I_term[6]/(3 * sqrt(2 *
#>          n * pi))
#>      coeff2 = (n - 1) * I_term[1]/24 - (n - 1) *
#>          (n + 2) * I_term[2]/(12 * n) + (n + 4) *
#>          (n - 1) * I_term[3]/(24 * n)
#>      coeff3 = (n - 1) * (2 * n + 5) * I_term[1]/72 -
#>          (n - 1) * (2 * n^2 + 5 * n + 8) * I_term[2]/(24 *
#>          n) + (n - 1) * (2 * n^2 + 5 * n + 12) *
#>          I_term[3]/(24 * n) - (n - 1) * (2 * n^2 +
#>          5 * n + 12) * I_term[4]/(72 * n)
#>      new_prob[t] = 1 - (I_term[1]/2 + S * coeff1 -
#>          K * coeff2 + S^2 * coeff3)
#>  }
#>      good_tscore = which(abs(new_prob - significance) <
#>          1e-04)
#>      new_tscore = tstat_list[good_tscore[which.min(abs(good_tscore -
#>          which.min(abs(tstat_list - closest_t))))]]
#>  }
#>  else {
#>      tstat_list = seq(from = (tscore - 3), to = (tscore +
#>          3), by = 1e-04)
#>      new_prob = c()
#>      for (t in 1:length(tstat_list)) {
#>          u0 = 1/(1 + (tstat_list[t]^2/(n - 1)))
#>          b = c(0.5, 0.5, 0.5, 0.5, 1, 1)
#>          w = b * (u0/(1 - u0))
#>          a = c((n - 1)/2, (n + 1)/2, (n + 3)/2, (n +
#>              5)/2, (n - 1)/2, (n + 1)/2)
#>          I_term = c()
#>          for (i in 1:6) {
#>              c1 = stats::pgamma(w[i], shape = a[i], rate = 1)
#>              c4 = (a[i] - 1 - w[i])/(2 * b[i])
#>              c5 = (a[i]^3/2 - 5 * a[i]^2/3 + 3 * a[i]/2 -
#>                  1/3)
#>              c6 = w[i] * (3 * a[i]^2/2 - 11 * a[i]/6 +
#>                  1/3)

```

```

#>         c7 = w[i]^2 * (3 * a[i]/2 - 1/6)
#>         if (n > 341) {
#>             c2 = Rmpfr::igamma(a[i], 0)
#>             c3 = ((exp(-w[i]) * w[i]^Rmpfr::mpfr(a[i],
#>                 128))/Rmpfr::igamma(a[i], 0))
#>             I_term[i] = Rmpfr::asNumeric(c1/c2 + c3 *
#>                 (c4 + (1/(2 * b[i]))^2) * (c5 - c6 + c7 -
#>                 w[i]^3/2)))
#>         }
#>         else {
#>             c2 = gamma(a[i])
#>             c3 = ((exp(-w[i]) * w[i]^a[i])/gamma(a[i]))
#>             I_term[i] = (c1/c2 + c3 * (c4 + (1/(2 *
#>                 b[i]))^2) * (c5 - c6 + c7 - w[i]^3/2)))
#>         }
#>     }
#>     coeff1 = (2 * n - 1) * I_term[5]/(6 * sqrt(2 *
#>         n * pi)) - (n - 1) * I_term[6]/(3 * sqrt(2 *
#>         n * pi))
#>     coeff2 = (n - 1) * I_term[1]/24 - (n - 1) *
#>         (n + 2) * I_term[2]/(12 * n) + (n + 4) *
#>         (n - 1) * I_term[3]/(24 * n)
#>     coeff3 = (n - 1) * (2 * n + 5) * I_term[1]/72 -
#>         (n - 1) * (2 * n^2 + 5 * n + 8) * I_term[2]/(24 *
#>         n) + (n - 1) * (2 * n^2 + 5 * n + 12) *
#>         I_term[3]/(24 * n) - (n - 1) * (2 * n^2 +
#>         5 * n + 12) * I_term[4]/(72 * n)
#>     new_prob[t] = 1 - (I_term[1]/2 + S * coeff1 -
#>         K * coeff2 + S^2 * coeff3)
#> }
#> good_tscore = which(abs(new_prob - significance) <
#>     1e-04)
#> if (length(good_tscore) > 0) {
#>     new_tscore = tstat_list[good_tscore[which.min(abs(good_tscore -
#>         which(tstat_list == tscore))]]]
#>     Skewed_UPL = mean(data_temp$emissions) + new_tscore *
#>         sqrt(var.s * (1/n + 1/future_runs))
#> }
#> else if (dplyr::between(significance, min(new_prob),
#>     max(new_prob))) {
#>     closest_t = tstat_list[which.min(abs(new_prob -
#>         significance))]
#>     reset_t1 = tstat_list[which.min(abs(new_prob -
#>         significance)) - 1]
#>     reset_t2 = tstat_list[which.min(abs(new_prob -
#>         significance)) + 1]
#>     tstat_list = seq(from = reset_t1, to = reset_t2,
#>         length.out = 5000)
#>     new_prob = c()
#>     for (t in 1:length(tstat_list)) {
#>         u0 = 1/(1 + (tstat_list[t]^2/(n - 1)))
#>         b = c(0.5, 0.5, 0.5, 0.5, 1, 1)
#>         w = b * (u0/(1 - u0))
#>         a = c((n - 1)/2, (n + 1)/2, (n + 3)/2, (n +

```

```

#>         5)/2, (n - 1)/2, (n + 1)/2)
#> I_term = c()
#> for (i in 1:6) {
#>     c1 = stats::pgamma(w[i], shape = a[i],
#>         rate = 1)
#>     c4 = (a[i] - 1 - w[i])/(2 * b[i])
#>     c5 = (a[i]^3/2 - 5 * a[i]^2/3 + 3 * a[i]/2 -
#>         1/3)
#>     c6 = w[i] * (3 * a[i]^2/2 - 11 * a[i]/6 +
#>         1/3)
#>     c7 = w[i]^2 * (3 * a[i]/2 - 1/6)
#>     if (n > 341) {
#>         c2 = Rmpfr::igamma(a[i], 0)
#>         c3 = ((exp(-w[i]) * w[i]^Rmpfr::mpfr(a[i],
#>             128))/Rmpfr::igamma(a[i], 0))
#>         I_term[i] = Rmpfr::asNumeric(c1/c2 +
#>             c3 * (c4 + (1/(2 * b[i])^2) * (c5 -
#>                 c6 + c7 - w[i]^3/2)))
#>     }
#>     else {
#>         c2 = gamma(a[i])
#>         c3 = ((exp(-w[i]) * w[i]^a[i])/gamma(a[i]))
#>         I_term[i] = (c1/c2 + c3 * (c4 + (1/(2 *
#>             b[i])^2) * (c5 - c6 + c7 - w[i]^3/2)))
#>     }
#> }
#> coeff1 = (2 * n - 1) * I_term[5]/(6 * sqrt(2 *
#>     n * pi)) - (n - 1) * I_term[6]/(3 * sqrt(2 *
#>     n * pi))
#> coeff2 = (n - 1) * I_term[1]/24 - (n - 1) *
#>     (n + 2) * I_term[2]/(12 * n) + (n + 4) *
#>     (n - 1) * I_term[3]/(24 * n)
#> coeff3 = (n - 1) * (2 * n + 5) * I_term[1]/72 -
#>     (n - 1) * (2 * n^2 + 5 * n + 8) * I_term[2]/(24 *
#>     n) + (n - 1) * (2 * n^2 + 5 * n + 12) *
#>     I_term[3]/(24 * n) - (n - 1) * (2 * n^2 +
#>     5 * n + 12) * I_term[4]/(72 * n)
#> new_prob[t] = 1 - (I_term[1]/2 + S * coeff1 -
#>     K * coeff2 + S^2 * coeff3)
#> }
#> good_tscore = which(abs(new_prob - significance) <
#>     1e-04)
#> new_tscore = tstat_list[good_tscore[which.min(abs(good_tscore -
#>     which.min(abs(tstat_list - closest_t))))]]
#> }
#> }
#> Skewed_UPL = emission_mean + new_tscore * sqrt(var.s *
#>     (1/n + 1/future_runs))
#> }
#> }
#> return(Skewed_UPL)
#> }
#> <bytecode: 0x000001df59b3ee68>
#> <environment: namespace:UPLforOAR>

```


obs_density()

```
#> function (data, emissions, xvals = NULL, up = Inf, low = 0, kernel = "gamma",
#>   bw = NULL)
#> {
#>   data_temp = tibble::tibble(emissions = data[[emissions]])
#>   sigma = stats::sd(data_temp$emissions)
#>   if (sigma == 0) {
#>     stop("Cannot find density for zero variance data")
#>   }
#>   if (is.null(bw)) {
#>     bw = sigma * nrow(data_temp)^(-2/5)
#>   }
#>   if (is.null(xvals)) {
#>     xvals = seq(0, 3 * max(data_temp$emissions), length.out = 1024)
#>   }
#>   if (is.numeric(bw)) {
#>     Obs_onPoint = np::npuniden.boundary(X = data_temp$emissions,
#>       Y = data_temp$emissions, a = low, b = up, proper = TRUE,
#>       kertype = kernel, h = bw)
#>     obs_den_df = np::npuniden.boundary(X = data_temp$emissions,
#>       Y = xvals, a = low, b = up, proper = TRUE, kertype = kernel,
#>       h = bw)
#>   }
#>   else if (is.character(bw)) {
#>     Obs_onPoint = np::npuniden.boundary(X = data_temp$emissions,
#>       Y = data_temp$emissions, bwmethod = bw, a = low,
#>       b = up, proper = TRUE, kertype = kernel)
#>     obs_den_df = np::npuniden.boundary(X = data_temp$emissions,
#>       Y = xvals, a = low, b = up, proper = TRUE, kertype = kernel,
#>       bwmethod = bw)
#>   }
#>   obs_den_df = tibble::tibble(x_hat = xvals, ydens = obs_den_df$y)
#>   Obs_onPoint = tibble::tibble(emissions = data_temp$emissions,
#>     ydens = Obs_onPoint$y)
#>   test_int = sfsmisc::integrate.xy(obs_den_df$x_hat, obs_den_df$ydens)
#>   if (abs(test_int - 1) > 0.05) {
#>     warning("density distribution does not integrate to 1,\n          consider adjusting xvals")
#>   }
#>   output = list(Obs_onPoint = Obs_onPoint, obs_den_df = obs_den_df)
#>   return(output)
#> }
```

#> <bytecode: 0x000001df5b11af60>
#> <environment: namespace:UPLforOAR>

make_environment()

```
#> function (data, HAP, emissions, sources, subcat_name = NA, subcat_level = NA,
#>   type = c("New", "Existing", "As-is"), CAA_section = 112,
#>   national_N = NA, meas_unit, future_runs = 3, significance = 0.99,
#>   ...)
#> {
#>   future_runs = as.integer(future_runs)
#>   if (!is.integer(future_runs)) {
#>     stop("future_runs must be a positive integer")
#>   }
#> }
```

```

#>   }
#>   if (future_runs < 1) {
#>     stop("future_runs must be a positive integer")
#>   }
#>   if (significance >= 1) {
#>     stop("significance must be greater then 0 and less than 1")
#>   }
#>   if (significance <= 0) {
#>     stop("significance must be greater then 0 and less than 1")
#>   }
#>   if (is.na(subcat_name)) {
#>     data_temp = tibble::tibble(emissions = data[[emissions]],
#>                               sources = data[[sources]])
#>   }
#>   else {
#>     data_temp = tibble::tibble(emissions = data[[emissions]],
#>                               sources = data[[sources]], subcat = data[[subcat_name]])
#>     if (!is.factor(data_temp$subcat)) {
#>       data_temp$subcat = factor(data_temp$subcat)
#>     }
#>     if (!(subcat_level %in% levels(data_temp$subcat))) {
#>       stop("subcat_level does not match levels of subcat_name")
#>     }
#>     data_temp = subset(data_temp, data_temp$subcat == subcat_level)
#>   }
#>   if (!is.numeric(data_temp$emissions)) {
#>     stop("Emissions must be numeric vector")
#>   }
#>   if ((!is.character(data_temp$sources)) & (!is.factor(data_temp$sources))) {
#>     stop("Sources must be a character or factor vector")
#>   }
#>   if (!is.character(emissions)) {
#>     emissions = colnames(data[emissions])
#>   }
#>   if (!is.character(sources)) {
#>     sources = colnames(data[sources])
#>   }
#>   analysis_env = list2env(list(HAP = HAP, emissions = emissions,
#>                               subcat_name = subcat_name, subcat_level = subcat_level,
#>                               data = data_temp, CAA_section = CAA_section, national_N = national_N,
#>                               meas_unit = meas_unit, sources = sources, type = type,
#>                               significance = significance, future_runs = future_runs,
#>                               more_args = ...), parent = as.environment(2))
#>   return(analysis_env)
#> }
#> <bytecode: 0x000001df55bb52a8>
#> <environment: namespace:UPLforOAR>

```

Appendix C: System and Package Versions

```
#> R version 4.4.0 (2024-04-24 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 22631)
#>
#> Matrix products: default
#>
#>
#> locale:
#> [1] LC_COLLATE=English_United States.utf8
#> [2] LC_CTYPE=English_United States.utf8
#> [3] LC_MONETARY=English_United States.utf8
#> [4] LC_NUMERIC=C
#> [5] LC_TIME=English_United States.utf8
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] grid      stats      graphics  grDevices utils      datasets  methods
#> [8] base
#>
#> other attached packages:
#> [1] rlang_1.1.6      knitr_1.50       UPLforOAR_0.1.0 sfsmisc_1.1-22
#> [5] np_0.60-18       EnvStats_3.1.0  lubridate_1.9.4 forcats_1.0.1
#> [9] stringr_1.5.2    purrr_1.1.0      readr_2.1.5      tidyr_1.3.1
#> [13] tibble_3.3.0     tidyverse_2.0.0 ggplot2_4.0.0    dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#> [1] generics_0.1.4    stringi_1.8.7     lattice_0.22-6     hms_1.1.3
#> [5] cubature_2.1.4    digest_0.6.37      magrittr_2.0.4     evaluate_1.0.5
#> [9] timechange_0.3.0  RColorBrewer_1.1-3 fastmap_1.2.0      Matrix_1.7-0
#> [13] survival_3.5-8    scales_1.4.0       cli_3.6.5          crayon_1.5.3
#> [17] bit64_4.6.0-1     splines_4.4.0      withr_3.0.2        yaml_2.3.10
#> [21] parallel_4.4.0    tools_4.4.0        SparseM_1.84-2     tzdb_0.5.0
#> [25] MatrixModels_0.5-4 boot_1.3-30         vctrs_0.6.5        R6_2.6.1
#> [29] lifecycle_1.0.4   bit_4.6.0          vroom_1.6.6        MASS_7.3-60.2
#> [33] pkgconfig_2.0.3    pillar_1.11.1      gtable_0.3.6       glue_1.8.0
#> [37] Rcpp_1.1.0         xfun_0.53          tidyselect_1.2.1    rstudioapi_0.17.1
#> [41] farver_2.1.2       patchwork_1.3.2     htmltools_0.5.8.1  labeling_0.4.3
#> [45] rmarkdown_2.30     compiler_4.4.0     quadprog_1.5-8     quantreg_6.1
#> [49] S7_0.2.0
```

Appendix D: Rmd Code Chunks

Main document code chunks:

```
# Below is an example of loading data, sorting and selecting to top performers,
# and calculating the correct distribution and UPL for existing source and NSPS.
# If there are multiple HAP or subcategories, simply repeat with code block for
# those data, making sure to use unique names so you don't overwrite the results.

if (params$CAA_section == 112){
  CAA_text = 'CAA Section 112(d)(3)'
} else if (params$CAA_section == 129){
  CAA_text = 'CAA Section 129(a)(2)'
}

# If using a single data set then upload with params:
dat_industry = read_csv(params$data)

# Or load manually:
dat_industry = read_csv('Example_data2.csv')
# If it is not in your working directory, you will need to include the file path:
# dat_industry = read_csv('USER/name/data/Example_data2.csv')

# In this example data, column 1 is the sources, there are two HAPS, Chemical X
# and Chemical Y in columns 2 and 3 in your data, named 'chemX' and 'chemY', and
# one subcategory, Size, with levels A and B.

# Let's suppose we want Existing source UPL calculations for both HAP's, the Size
# subcategory only applies to one HAP (Chemical X), and we also want New source on
# subcategory B. This is four analyses: existing source UPL's for Chemical X
# subcategory A & B, and Chemical Y, and new source UPL for Chemical X subcategory B.

# Next, designate lists (of length four, the number of calculations) for the
# emissions, sources, and subcategory for which
# you will be doing each UPL calculation, and whether or not it is for an existing
# or new source, or as-is (no subsetting):
subcat_list = c('Size', 'Size', 'Size', NA)
subcat_levels = c('A', 'B', 'B', NA)
type_list = c('Existing', 'Existing', 'New', 'Existing')
HAP_list = c('Chemical X', 'Chemical X', 'Chemical X', 'Chemical Y')
emiss_list = c('chemX', 'chemX', 'chemX', 'chemY')
meas_unit_list = c('ppm', 'ppm', 'ppm', 'mg/m3/s')
sources = 1

# Note that HAP_list are the names as we want them in text or labels
# the lists for emissions, sources, and subcat_list can be the variable names
# as a string or indicate the column number in the data set.
# For example, let's say your sources are in column 1 and are named 'boiler units'
# R doesn't like spaces and will replace them (and special characters) with periods.
# We can designate the sources by name ('boiler.units') or position (1).

# We will do the default 3 future runs and 0.99 significance for all four UPLs:
future_run_list = c(3, 3, 3, 3)
sig_list = c(0.99, 0.99, 0.99, 0.99)
```

```

# Now we can create a list of environments to populate the .Rmd document:
env_list = list()
for (i in 1:length(HAP_list)){
  env_temp = make_environment(data = dat_industry, HAP = HAP_list[i],
                             emissions = emiss_list[i],
                             sources = sources,
                             subcat_name = subcat_list[i],
                             subcat_level = subcat_levels[i],
                             type = type_list[i],
                             meas_unit = meas_unit_list[i],
                             CAA_section = params$CAA_section,
                             national_N = params$national_N,
                             future_runs = future_run_list[i],
                             significance = sig_list[i])
  env_list = c(env_list, env_temp)
}

# You also use the make_environment() function outside of the for loop

# Run the analysis for each environment set:
for (i in 1:length(env_list)){
  cat(asis_output(knit_child('Example_report_children/Analysis_template.Rmd',
                             envir = env_list[[i]], quiet = TRUE)))
}
col1=c('Subcategory 1','Subcategory 2','Subcategory 3','Subcategory 4',
       'Subcategory 5')
col2=c('HAP 1, HAP 2', 'HAP 1', 'HAP 1, HAP 2, HAP 3', 'None', 'None')
col3=c('HAP 1, HAP 2', 'HAP 1', 'HAP 1, HAP 2, HAP 3', 'None', 'None')
col4=c('HAP 1, HAP 2, X, Y, Z', 'HAP 1, X, Y, Z', 'HAP 1, HAP 2, HAP 3, X, Y, Z',
       'None', 'X, Y, Z')
table1=tibble(col1=col1,col2=col2,col3=col3,col4=col4)
table1%>%knitr::kable(
  caption="Here is an example of a table in case you need one",
  col.names=c(' ',
              'Existing MACT floor (old units)',
              'Existing MACT floor (new units)',
              'New MACT floor'))
# check if any UPL results from new and existing source pairs need to be set
# such that the new UPL <= existing UPL.
order_combn = combn(seq(1,length(HAP_list), by = 1), 2)
HAP_combn = combn(HAP_list, 2)
subcat_combn = combn(subcat_list, 2)
level_combn = combn(subcat_levels, 2)
meas_combn = combn(meas_unit_list, 2)

check_HAPs = HAP_combn[1, ] == HAP_combn[2, ]
check_subcat = subcat_combn[1, ] == subcat_combn[2, ]
check_levels = level_combn[1, ] == level_combn[2, ]
check_units = meas_combn[1, ] == meas_combn[2, ]
for (i in 1:length(check_HAPs)){
  if (all(check_HAPs[i], check_subcat[i], check_levels[i],check_units[i])){
    pair1 = order_combn[1,i]
    pair2 = order_combn[2,i]

```

```

type1 = env_list[[pair1]]$type
type2 = env_list[[pair2]]$type
UPL1 = env_list[[pair1]]$UPL
UPL2 = env_list[[pair2]]$UPL
if ((type1 == "New") & (UPL1 > UPL2) & (type2 == 'Existing')){
  env_list[[pair1]]$UPL_old = UPL1
  env_list[[pair1]]$UPL = UPL2
  env_list[[pair1]]$UPL_warn = TRUE
} else if ((type2 == "New") & (UPL2 > UPL1) & (type1 == 'Existing')){
  env_list[[pair2]]$UPL_old = UPL2
  env_list[[pair2]]$UPL = UPL1
  env_list[[pair2]]$UPL_warn = TRUE
}
}
}
UPL_list = c()
for (i in 1:length(env_list)){
  UPL_temp = signif(env_list[[i]]$UPL, 4)
  UPL_list = c(UPL_list, UPL_temp)
}
dat_UPL = tibble(HAP = HAP_list,
                  Subcategory = subcat_list,
                  Level = subcat_levels,
                  Type = type_list,
                  UPL = UPL_list,
                  Sig = sig_list,
                  Runs = future_run_list)
dat_UPL %>% knitr::kable(
  caption = "Upper Predictive Limit (UPL) results for MACT floors in
(measurement units)", escape = FALSE,
  col.names = c("HAP", "Subcategory<br/>Name", "Subcategory<br/>Level", "Source Type",
                "UPL", "Significance", "No. Future<br/>Runs"))

```

Child document code chunks:

Analysis_template.Rmd

```

````{r, echo = FALSE, appendix = FALSE}
if (type == 'Existing'){
 df = data
 if (any(df$emissions == 0)){
 zero_warn = TRUE
 } else {
 zero_warn = FALSE
 }
 df = MACT_existing(CAA_section = CAA_section, data = df,
 emissions = 'emissions', sources = 'sources',
 national_N = national_N)
 df_mean = df %>% group_by(sources) %>%
 summarize(means = mean(emissions),
 counts = n())

 if (all(df_mean$counts == 1)){
 source_fig = FALSE
 } else {
 source_fig = TRUE
 }
}

```

```

}
} else if (type == "New"){
 df = data
 if (any(df$emissions == 0)){
 zero_warn = TRUE
 } else {
 zero_warn = FALSE
 }
 source_fig = FALSE
 df = MACT_new(data = df, emissions = 'emissions', sources = 'sources')
} else if (type == "As-is"){
 df = data
 if (any(df$emissions == 0)){
 zero_warn = TRUE
 } else {
 zero_warn = FALSE
 }
 df_mean = df %>% group_by(sources) %>%
 summarize(means = mean(emissions),
 counts = n())

 if (all(df_mean$counts == 1)){
 source_fig = FALSE
 } else {
 source_fig = TRUE
 }
}
UPL_warn = FALSE
best_source = as.character(df$sources)[1]
df$ln_emiss = log(df$emissions)
df$ln_emiss = replace(df$ln_emiss, !is.finite(df$ln_emiss), NA)
distribution_result = distribution_type(data = df, emissions = 'emissions')
if (distribution_result == "Normal"){
 UPL = Normal_UPL(data = df,
 emissions = 'emissions',
 significance = significance,
 future_runs = future_runs)
} else if (distribution_result == "Lognormal"){
 UPL = Lognormal_UPL(data = df,
 emissions = 'emissions',
 significance = significance,
 future_runs = future_runs)
} else if (distribution_result == "Skewed"){
 UPL = Skewed_UPL(data = df,
 emissions = 'emissions',
 significance = significance,
 future_runs = future_runs)
}
if (!is.na(subcat_name)){
 subcat_text = paste0('for ', subcat_name, ' subcategory ', subcat_level)
} else {
 subcat_text = ''
}
...

```

Source\_variance.Rmd

```
`r type` Source `r HAP` `r subcat_text`
```

The final data set for the `r HAP` `r type` source MACT floor `r subcat\_text` included  
→ `r nrow(df)` test runs from `r length(unique(df\$sources))` sources. The best  
→ performer for the new source calculation was `r best\_source` with an average emission  
→ of `r signif(mean(df\$emissions), digits = 4)` (`r meas\_unit`) from `r nrow(df)` test  
→ runs.

```
`{r, echo = FALSE, fig.align = "center", fig.height = 4, appendix = FALSE, eval =
→ source_fig, fig.cap = "Observation density of emissions by source type. Source
→ averages are indicated by vertical lines."}
if (!is.na(subcat_name)){
 ggplot(data = df, aes(emissions, fill = sources))+
 geom_density(alpha = 0.5, bounds = c(0, Inf))+
 ylab("Density")+xlab(paste0(HAP, " emissions (" , meas_unit, ")"))+
 ggtitle(paste0(subcat_name, " ", subcat_level, " ", type,
 " source test runs"))+
 multi_source_theme()+labs(fill = 'Top sources',
 color = 'Top sources')+
 geom_vline(aes(xintercept = means, color = sources),
 data = df_mean, linewidth = 1)+
 guides(fill = guide_legend(nrow = 2, byrow = TRUE),
 color = guide_legend(nrow = 2, byrow = TRUE))+
 scale_x_continuous(expand = expansion(mult = c(0, 0.05)))+
 scale_y_continuous(expand = expansion(mult = c(0, 0.05)))+
 coord_cartesian(clip = 'off')+
 geom_rug(sides = 'b', aes(x = emissions,
 color = sources),
 data = df, alpha = 0.5, outside = TRUE)
} else {
 ggplot(data = df, aes(emissions, fill = sources))+
 geom_density(alpha = 0.5, bounds = c(0, Inf))+
 ylab("Density")+xlab(paste0(HAP, " emissions (" , meas_unit, ")"))+
 ggtitle(paste0(type, " source test run densities"))+
 multi_source_theme()+labs(fill = 'Top sources',
 color = 'Top sources')+
 geom_vline(aes(xintercept = means, color = sources),
 data = df_mean, linewidth = 1)+
 guides(fill = guide_legend(nrow = 2, byrow = TRUE),
 color = guide_legend(nrow = 2, byrow = TRUE))+
 scale_x_continuous(expand = expansion(mult = c(0, 0.05)))+
 scale_y_continuous(expand = expansion(mult = c(0, 0.05)))+
 coord_cartesian(clip = 'off')+
 geom_rug(sides = 'b', aes(x = emissions,
 color = sources),
 data = df, alpha = 0.5, outside = TRUE)
}
`{r`
```

Analysis\_results.Rmd

```
`r type` Source `r HAP` `r subcat_text`
```



```

```{r, eval = UPL_warn, echo = FALSE, results = 'asis'}
cat(paste0("The ", HAP, " ", subcat_text, " UPL calculated for the New source data set, ",
  ↳ signif(UPL_old, 3), " was larger than the UPL for the Existing source data set.
  ↳ Therefore, the Existing source UPL was used instead."))
```

```

The distribution that best represents the ``r type`` source data set for ``r HAP`` ``r subcat_text`` is the ``r distribution_result`` distribution, with a corresponding UPL of ``r signif(UPL, digits = 4)`` (``r meas_unit``).

```

```{r, echo = FALSE, appendix = FALSE, fig.dim = c(5, 4), fig.align = "center", fig.cap =
  ↳ "Observation density of emissions for overall population of test runs. The obseration
  ↳ data are indicated in black as points and a rug along the axis, with the observation
  ↳ density distribution as a black line. The fitted distribution that is the basis of
  ↳ the UPL estimate is colored orange. The average of the emission test runs is the
  ↳ vertical black line and the UPL result is the vertical orange line."}
if (!exists('xvals')){
  xvals = seq(0,3*max(df$emissions), length.out = 1024)
} else if (is.na(xvals)){
  xvals = seq(0,3*max(df$emissions), length.out = 1024)
}
if (!exists('up')){
  up = Inf
} else if (is.na(up)){
  up = Inf
}
if (!exists('low')){
  low = 0
} else if (is.na(low)){
  low = 0
}
if (!exists('kernel_arg')){
  kernel_arg = 'gamma'
} else if (is.null(kernel_arg)){
  kernel_arg = 'gamma'
} else if (is.na(kernel_arg)){
  kernel_arg = 'gamma'
}
if (!exists('bw')){
  bw = NULL
} else if (!is.null(bw)){
  if (is.na(bw)){
    bw = NULL
  }
}
obs_dens_results = obs_density(df, xvals = xvals, low = low, up = up,
  bw = bw, kernel = kernel_arg,
  emissions = 'emissions')
Obs_onPoint = obs_dens_results$Obs_onPoint
obs_den_df = obs_dens_results$obs_den_df
if (distribution_result == "Normal"){
  pdf_n = dnorm(obs_den_df$x_hat, mean = mean(df$emissions, na.rm = T),
    sd = sd(df$emissions, na.rm = T))
  pred_dat = tibble(x_hat = obs_den_df$x_hat, pdf_n)
}

```

```

} else if (distribution_result == "Lognormal"){
  pdf_n = dlnorm(obs_den_df$x_hat, mean = log(mean(df$emissions, na.rm = T)),
    sd = sd(log(df$emissions), na.rm = T))
  pred_dat = tibble(x_hat = obs_den_df$x_hat, pdf_n)
} else {
  pred_dat = tibble(x_hat = rep(NA, 1024), pdf_n = rep(NA, 1024))
}
if (!is.na(subcat_name)){
  ggplot()+
    geom_line(data = obs_den_df, size = 0.75,
      aes(y = ydens, x = x_hat, color = 'a'))+
    geom_area(data = obs_den_df, alpha = 0.25,
      aes(y = ydens, x = x_hat, fill = 'a'), show.legend = FALSE)+
    geom_point(aes(y = ydens, x = emissions), data = Obs_onPoint,
      size = 3, alpha = 0.5, shape = 19, color = 'black')+
    geom_line(aes(y = pdf_n, x = x_hat, color = 'b'),
      data = pred_dat, size = 0.75, linetype = 2)+
    geom_area(aes(y = pdf_n, x = x_hat, fill = 'b'), alpha = 0.25,
      data = pred_dat, show.legend = FALSE)+
    ylab("Density")+xlab(paste0(HAP, " emissions (", meas_unit, ")"))+
    ggtitle(paste0(subcat_name, " ", subcat_level, " ", type,
      " source\nMACT floor test run population"))+
    pop_distr_theme()+
    geom_vline(aes(xintercept = mean(df$emissions),
      color = 'a'), linewidth = 1, linetype = 1)+
    geom_vline(aes(xintercept = UPL, color = 'b'),
      linewidth = 1, linetype = 2)+
    scale_x_continuous(expand = expansion(mult = c(0, 0.05)))+
    scale_y_continuous(expand = expansion(mult = c(0, 0.05)))+
    coord_cartesian(clip = 'off')+
    labs(color = 'Distribution:')+
    geom_rug(sides = 'b', aes(x = emissions), data = df,
      alpha = 0.5, outside = TRUE, color = 'black')+
    scale_color_manual(values = c('black', '#FF7F00'),
      labels = c('Observations', distribution_result))+
    scale_fill_manual(values = c('black', '#FF7F00'),
      labels = c('Observations', distribution_result))
} else {
  ggplot()+
    geom_line(data = obs_den_df, size = 0.75,
      aes(y = ydens, x = x_hat, color = 'a'))+
    geom_area(data = obs_den_df, alpha = 0.25, show.legend = FALSE,
      aes(y = ydens, x = x_hat, fill = 'a'))+
    geom_point(aes(y = ydens, x = emissions), data = Obs_onPoint,
      size = 3, alpha = 0.5, shape = 19, color = 'black')+
    geom_line(aes(y = pdf_n, x = x_hat, color = 'b'),
      data = pred_dat, size = 0.75, linetype = 2)+
    geom_area(aes(y = pdf_n, x = x_hat, fill = 'b'), alpha = 0.25,
      data = pred_dat, show.legend = FALSE)+
    ylab("Density")+xlab(paste0(HAP, " emissions (", meas_unit, ")"))+
    ggtitle(paste0(type,
      " source\nMACT floor test run population"))+
    pop_distr_theme()+
    geom_vline(aes(xintercept = mean(df$emissions),

```

```

        color = 'a'), linewidth = 1, linetype = 1)+
geom_vline(aes(xintercept = UPL, color = 'b'),
           linewidth = 1, linetype = 2)+
scale_x_continuous(expand = expansion(mult = c(0, 0.05)))+
scale_y_continuous(expand = expansion(mult = c(0, 0.05)))+
coord_cartesian(clip = 'off')+
labs(color = 'Distribution:')+
geom_rug(sides = 'b', aes(x = emissions), data = df,
         alpha = 0.5, outside = TRUE, color = 'black')+
scale_color_manual(values = c('black', '#FF7F00'),
                  labels = c('Observations', distribution_result))+
scale_fill_manual(values = c('black', '#FF7F00'),
                 labels = c('Observations', distribution_result))
}
if (is.null(bw)){
  bw_result = sd(df$emissions) * nrow(df)^(-2/5)
} else if (is.character(bw)){
  raw_dens = npuniden.boundary(X = df$emissions, Y = xvals,
                             a = low, b = up, proper = TRUE,
                             kertype = kernel_arg, bwmethod=bw)

  bw_result = raw_dens$h
} else if (is.numeric(bw)){
  bw_result = bw
}
...

```

The density plot above of emission test run data was created using `ggplot2` in `R`. The
 → density curves were generated using `obs_density()` in `UPLforOAR` version `r`
 → `packageVersion('UPLforOAR')`, using a `r kernel_arg` kernel type, bounds of (`r`
 → `c(low, up)`), and a bandwidth of `r signif(bw_result, 3)`. For further details see
 → the `R` code in Appendix B.

Appendix_data.Rmd

```

`r type` source MACT floor analysis data set for HAP `r HAP` `r subcat_text`:

```{r, echo = FALSE, appendix = FALSE}
col_names = c(emissions, sources, 'log emissions')
df %>% knitr::kable(
 caption = "Top performing source emissions data used in MACT floor
 UPL calculations.", digits = 12, col.names = col_names)
...

```